

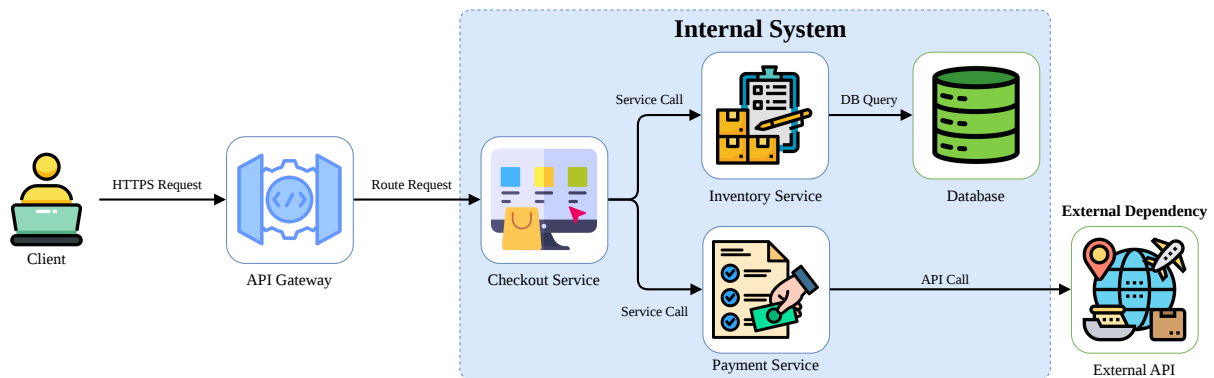
Observability Engineering Metrics, Logs, and Traces

Trần Quang Minh, Nguyễn Mạnh Khiêm (PhD)

Giới thiệu

Các hệ thống phần mềm hiện tại gồm nhiều thành phần như application, service, database, cache, message queue và dịch vụ bên thứ ba. Các thành phần này phụ thuộc lẫn nhau và cùng tham gia xử lý một request. Vì vậy, khi latency hoặc error rate tăng, engineer mới chỉ biết hệ thống có dấu hiệu bất thường, chưa biết nguyên nhân xuất hiện ở đâu trên request path.

Monitoring cung cấp thông tin về tài nguyên, hiệu năng và cảnh báo khi hệ thống vượt ngưỡng, qua đó giúp engineer nhận biết dấu hiệu ban đầu. Tuy nhiên, tín hiệu được ghi nhận tại một service không nhất thiết phản ánh nguồn gốc sự cố. Chẳng hạn, một request checkout thất bại có thể liên quan đến Checkout Service, downstream service, database hoặc external API.



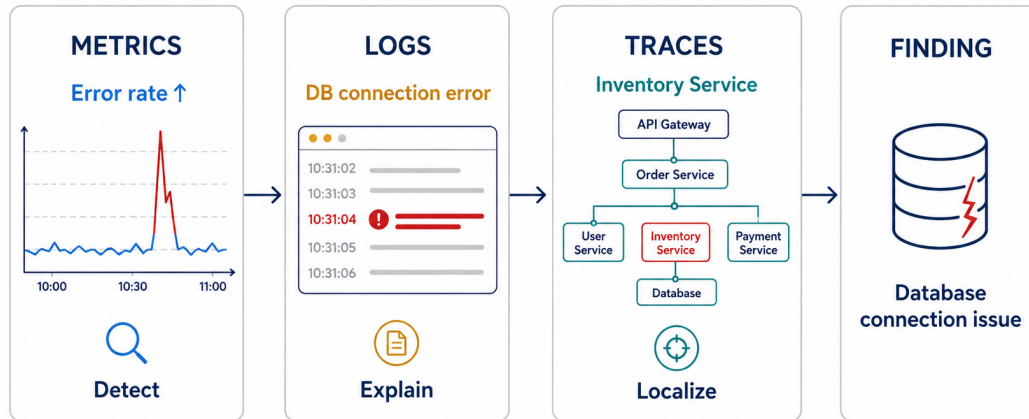
Hình 1: Một request đi qua nhiều dependency nên nguồn sự cố có thể nằm ngoài service được quan sát.

Những hệ thống này tạo ra nhiều loại dữ liệu vận hành như metrics, logs, traces, alerts và deployment events. Trong thực tế, các tổ chức thường kết hợp nhiều công cụ để thu thập và phân tích những dữ liệu đó. Theo [Grafana Labs Observability Survey 2025](#), các tổ chức sử dụng trung bình tám công nghệ observability và 39% xem complexity hoặc operational overhead là trở ngại lớn nhất. Kết quả cho thấy chuẩn hóa và liên kết dữ liệu từ nhiều nguồn là yêu cầu quan trọng trong vận hành.

Đây là vai trò của Observability: giúp hiểu trạng thái bên trong của hệ thống thông qua những dữ liệu mà hệ thống tạo ra. Observability Engineering tập trung vào việc thiết kế, thu thập và

liên kết các dữ liệu đó để engineer có thể đi từ một dấu hiệu bất thường đến những bằng chứng liên quan. Mục tiêu không phải là thu thập càng nhiều dữ liệu càng tốt, mà là đảm bảo dữ liệu có đủ context để hỗ trợ việc theo dõi và điều tra sự cố.

Observability thường sử dụng ba loại telemetry chính: Metrics, Logs và Traces. Metrics phản ánh trạng thái của hệ thống qua các giá trị số theo thời gian, Logs ghi lại những sự kiện đã xảy ra, còn Traces mô tả hành trình của request qua các thành phần trong hệ thống. Mỗi loại cung cấp một góc nhìn khác nhau và trở nên hữu ích hơn khi được phân tích cùng nhau.



Hình 2: Metrics phát hiện dấu hiệu bất thường, Logs cung cấp bằng chứng về sự kiện đã xảy ra, còn Traces xác định vị trí vấn đề trong request path.

Giá trị của observability không chỉ nằm ở từng loại telemetry riêng lẻ mà còn ở khả năng correlation giữa chúng. Nếu Metrics, Logs và Traces sử dụng metadata nhất quán, engineer có thể chuyển từ một metric bất thường đến đúng logs và trace liên quan mà không phải tìm kiếm trên toàn bộ hệ thống. Đây cũng là nền tảng để xây dựng dashboard, alert và quy trình điều tra incident hiệu quả hơn.

Khi lượng telemetry tăng, việc theo dõi và phân tích thủ công từng tín hiệu trở nên khó khăn hơn. Đây là lúc các phương pháp phân tích nâng cao có thể hỗ trợ engineer khai thác dữ liệu hiệu quả hơn. Bài tiếp theo sẽ đi sâu vào AIOps và cách tiếp cận này hỗ trợ phát hiện bất thường, liên kết các tín hiệu liên quan và ưu tiên vấn đề cần xử lý.

Mục lục

Giới thiệu	1
1 Fundamentals of Observability and System Signals	4
1.1 Monitoring and Observability	4
1.2 Metrics, Logs, and Traces	6
1.3 Choosing What to Measure	8
1.4 Service Reliability with SLI, SLO, and Error Budget	10
1.5 Hands-on Environment Setup	12
2 Metrics Collection and Analysis with Prometheus	14
2.1 Prometheus and the Metrics Pipeline	14
2.2 Exposing Metrics: Application Instrumentation and Exporters	17
2.3 Prometheus Time-Series Data Model	20
2.4 Hands-on: From Raw Metrics to Operational Signals with PromQL	23
2.5 Visualizing Metrics with Grafana	30
3 Logs with Loki and Alloy	33
3.1 From Metrics to Logs	33
3.2 Understanding Log Data	34
3.3 Collecting Logs with Grafana Alloy	36
3.4 Storing Logs with Loki	38
3.5 Hands-on: Investigating Checkout Errors with LogQL	40
4 Traces with OpenTelemetry and Jaeger	48
4.1 From Logs to Distributed Tracing	48
4.2 Trace Relationships and Context Propagation	51
4.3 Instrumenting Applications with OpenTelemetry	54
4.4 Trace Pipeline with OTEL Collector and Jaeger	56

1 Fundamentals of Observability and System Signals

1.1 Monitoring and Observability

Một hệ thống có thể vẫn đang chạy nhưng không hoạt động đúng như kỳ vọng. Application có thể phản hồi chậm, một số request có thể thất bại hoặc chỉ một nhóm người dùng gặp lỗi. Vì vậy, engineer cần theo dõi không chỉ trạng thái hoạt động mà còn cả những thay đổi có thể ảnh hưởng đến hiệu suất và trải nghiệm người dùng.

Monitoring là gì?

Monitoring là quá trình theo dõi liên tục những tín hiệu và điều kiện đã được xác định trước để đánh giá tình trạng của hệ thống. Engineer thường bắt đầu bằng những câu hỏi cụ thể như:

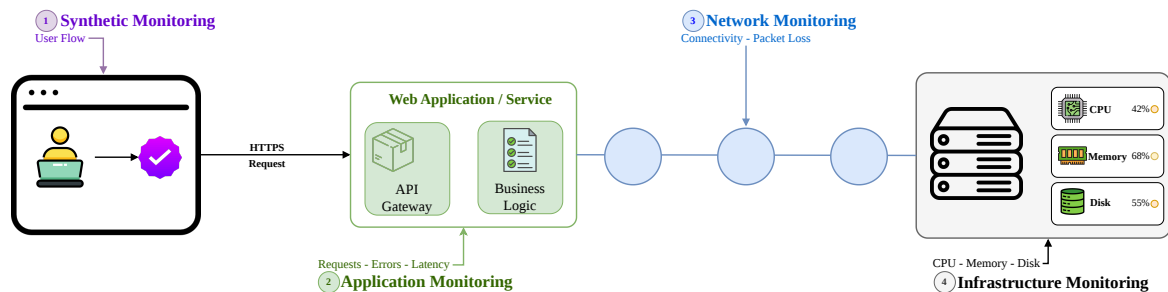
- Service có đang hoạt động không?
- CPU hoặc memory có vượt ngưỡng không?
- Request có đang phản hồi chậm không?
- Tỷ lệ request thất bại có tăng không?

Từ những câu hỏi này, team lựa chọn phép đo phù hợp và thiết lập điều kiện cảnh báo. Threshold là ngưỡng dùng để xác định khi nào một giá trị cần được xem là bất thường. Chẳng hạn, hệ thống có thể tạo alert nếu thời gian phản hồi của checkout vượt 500 ms liên tục trong 5 phút.

Input của monitoring là các tín hiệu vận hành đã được lựa chọn. Monitoring system thu thập những tín hiệu này, theo dõi sự thay đổi theo thời gian và so sánh chúng với các điều kiện đã đặt. Output có thể là một giá trị trên dashboard, trạng thái của service hoặc alert yêu cầu engineer chú ý.

Monitoring có thể được triển khai ở nhiều lớp khác nhau của hệ thống. Một số phạm vi phổ biến gồm:

- **Infrastructure Monitoring:** theo dõi CPU, memory, disk và các tài nguyên hạ tầng.
- **Application Monitoring:** theo dõi request, error, latency và hành vi của application.
- **Network Monitoring:** theo dõi kết nối và lưu lượng giữa các thành phần.
- **Synthetic Monitoring:** mô phỏng định kỳ hành vi người dùng bằng request hoặc script để kiểm tra hệ thống.



Hình 3: Monitoring được triển khai trên nhiều lớp của hệ thống.

Ngoài các phạm vi trên, monitoring còn có thể tập trung vào dependencies và third-party integrations, hiệu suất website, trải nghiệm của người dùng thật, hoạt động kinh doanh hoặc các dấu hiệu bảo mật. Cách phân loại có thể khác nhau vì một monitoring solution thường cung cấp đồng thời nhiều góc nhìn.

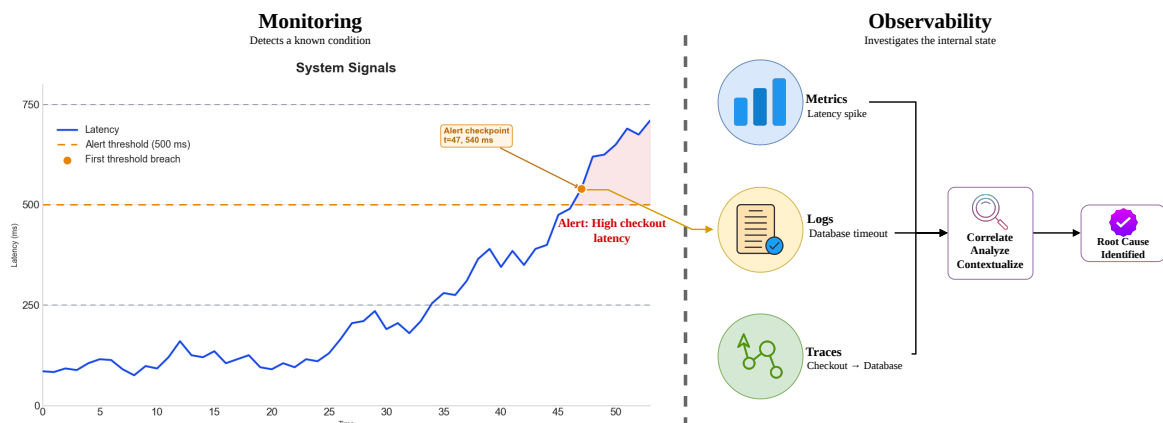
Observability là gì?

Monitoring có thể phát hiện checkout đang chậm, nhưng thông tin đó chưa cho biết nguyên nhân nằm ở application, database, network hay external service. Để tiếp tục điều tra, engineer cần khả năng nhìn sâu hơn vào trạng thái bên trong hệ thống, đây chính là vai trò của **Observability**. Observability là khả năng hiểu trạng thái bên trong thông qua những dữ liệu mà hệ thống tạo ra trong quá trình hoạt động. Thay vì chỉ kiểm tra các điều kiện đã định nghĩa trước, observability cho phép engineer đặt thêm những câu hỏi mới trong lúc điều tra, chẳng hạn:

- Vì sao request bị chậm?
- Thành phần nào bắt đầu phát sinh lỗi?
- Một dependency có đang ảnh hưởng đến application không?
- Vì sao lỗi chỉ xuất hiện với một số request?

Quan hệ giữa Monitoring và Observability

Monitoring và Observability được sử dụng cùng nhau trong quá trình vận hành. Monitoring theo dõi các điều kiện đã biết và tạo điểm bắt đầu khi phát hiện dấu hiệu bất thường. Từ đó, observability cung cấp dữ liệu và context để engineer tiếp tục điều tra những gì đang xảy ra bên trong hệ thống.



Hình 4: Từ monitoring alert đến quá trình điều tra bằng observability.

Trong ví dụ trên, Monitoring tạo alert High Checkout Latency khi checkout response time vượt 500 ms trong 5 phút. Từ điểm bắt đầu này, Observability giúp engineer kết hợp dữ liệu trong cùng khoảng thời gian, liên hệ các request chậm với lỗi database connection timeout và thu hẹp vấn đề về phía database connection.

1.2 Metrics, Logs, and Traces

Một request có thể đi qua nhiều thành phần trước khi hoàn tất. Nếu checkout đột nhiên chậm, engineer cần biết hệ thống có dấu hiệu bất thường nào, sự kiện gì đã xảy ra và request bị chậm tại đâu. Để điều tra, engineer sử dụng *telemetry*, tức dữ liệu được hệ thống tạo ra trong quá trình hoạt động. Ba loại telemetry chính là Metrics, Logs và Traces.

Metrics: Giá trị số theo thời gian

Để theo dõi trạng thái của hệ thống, engineer cần những giá trị có thể đo lường và so sánh theo thời gian. Mỗi metric biểu diễn một trạng thái hoặc hành vi cụ thể, đồng thời chứa tên, giá trị, timestamp và các label dùng để phân biệt nguồn dữ liệu.

Ví dụ, application có thể ghi nhận: `http_request_duration_seconds{service_name="checkout"} 0.82`

Trong đó:

- `http_request_duration_seconds` là tên metric, biểu thị thời gian xử lý request.
- `service_name="checkout"` xác định metric thuộc checkout service.
- 0.82 là giá trị đo được, tương ứng 0.82 giây.

Metrics phù hợp để theo dõi xu hướng và phát hiện các dấu hiệu như latency tăng, error rate vượt ngưỡng hoặc CPU usage bất thường.

Logs: Bản ghi sự kiện

Metrics cho thấy trạng thái hệ thống đang thay đổi, nhưng không cung cấp đầy đủ chi tiết về từng sự kiện. Một log entry ghi lại sự kiện tại một thời điểm cụ thể và có thể chứa timestamp, log level, service, error message cùng các thông tin liên quan.

Ví dụ:

```
{
  "timestamp": "2026-08-15T10:05:23Z",
  "level": "ERROR",
  "service.name": "checkout",
  "trace_id": "7f31a9",
  "message": "Database connection timeout"
}
```

Trong log entry này:

- `timestamp` cho biết thời điểm sự kiện xảy ra.
- `level` thể hiện mức độ của sự kiện. Giá trị **ERROR** cho biết operation đã gặp lỗi.
- `service.name` xác định service tạo ra log.
- `trace_id` là mã nhận diện request mà sự kiện thuộc về.
- `message` mô tả sự kiện đã xảy ra.

Logs cung cấp thông tin chi tiết về các sự kiện như request không hợp lệ, kết nối database thất bại hoặc external service trả về lỗi.

Traces: Hành trình của request

Trong hệ thống có nhiều service, các sự kiện của cùng một request có thể xuất hiện ở nhiều nơi khác nhau. Một trace kết nối các operation thuộc cùng request để thể hiện toàn bộ hành trình xử lý. Mỗi trace gồm nhiều span, trong đó mỗi span đại diện cho một operation như gọi API, truy vấn database hoặc gửi request đến external service.

Ví dụ, một checkout request có thể tạo ra trace như sau:

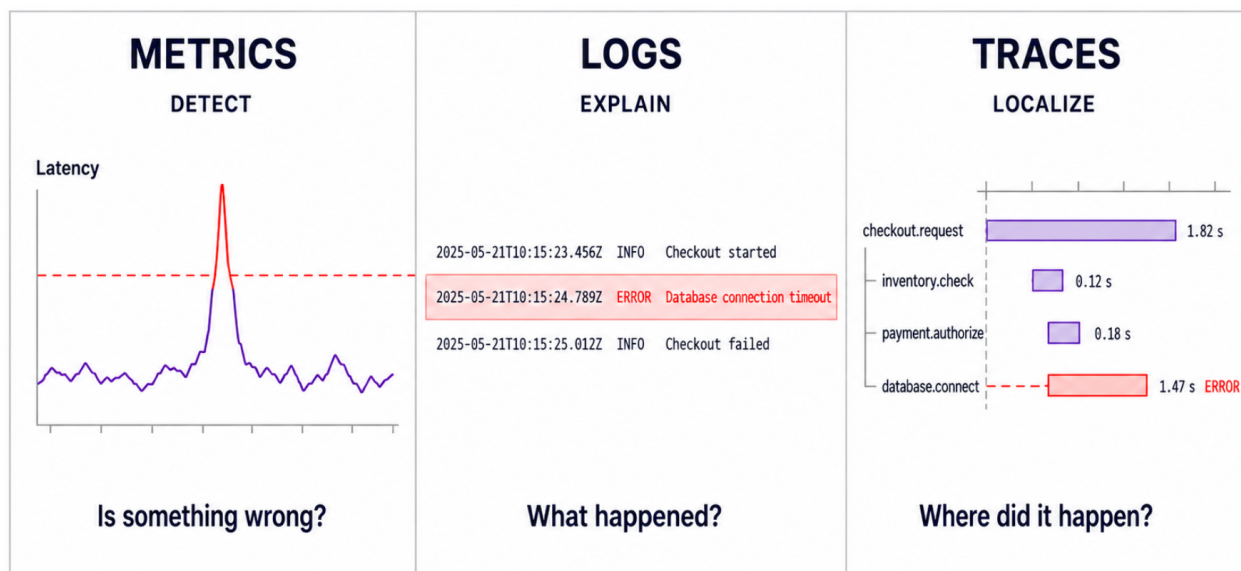
```
Trace ID: 7f31a9

GET /checkout 1.52 s ERROR
|__ inventory-service 1.50 s ERROR
|   |__ inventory.check_stock 1.49 s ERROR
|       |__ database.connect 1.47 s ERROR
```

Các span của cùng một request sử dụng chung Trace ID. Thời lượng bên cạnh mỗi span cho biết operation tương ứng mất bao lâu. Trong ví dụ này, `database.connect` chiếm phần lớn thời gian xử lý, khiến `inventory-service` và request `/checkout` kết thúc với trạng thái **ERROR**.

Liên kết Metrics, Logs và Traces

Correlation là quá trình liên kết các telemetry signal trong cùng một ngữ cảnh điều tra.



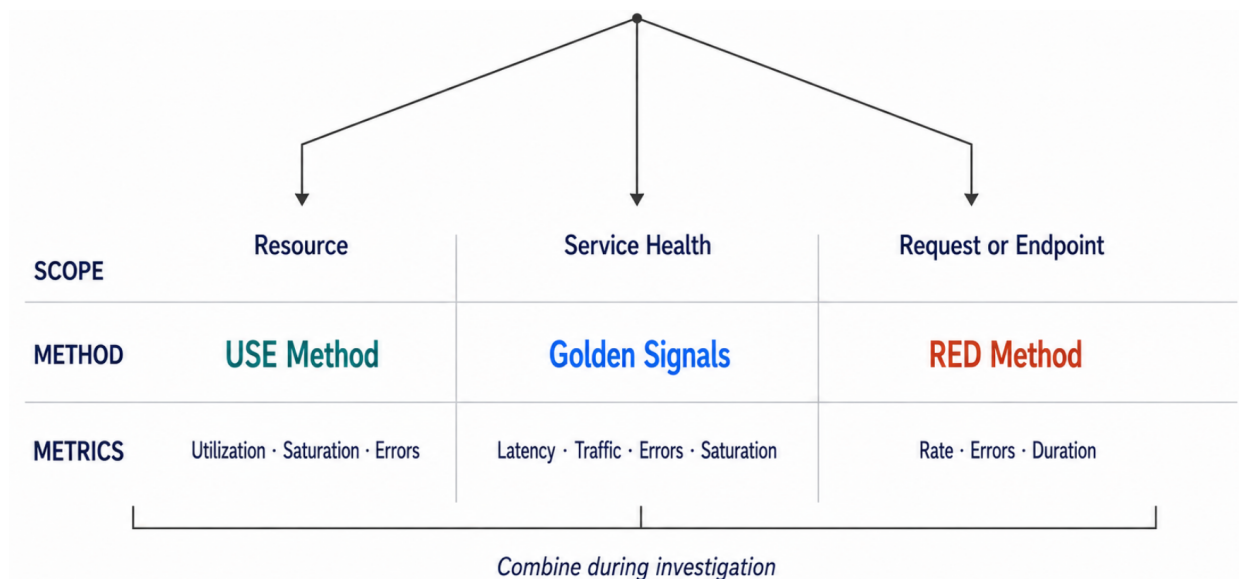
Hình 5: Từ latency spike trong Metrics, engineer sử dụng service và time window để tìm error log, sau đó dùng `trace_id` để mở trace và xác định span gây chậm.

Trong ví dụ ở Hình 5, metrics cho thấy p95 latency của checkout service tăng từ 300 ms lên 1.8 giây trong khoảng thời gian từ 10:00 đến 10:10. P95 latency là mức latency mà 95% request không vượt quá. Khi kiểm tra logs của cùng service trong khoảng thời gian này, engineer phát hiện nhiều lỗi `Database connection timeout`. Trường `trace_id` trong một log liên quan dẫn đến trace có span `database.connect` mất 1.47 giây, chiếm phần lớn tổng thời gian xử lý request.

Quá trình correlation sử dụng `service.name` để xác định service cần kiểm tra và time window để giới hạn các sự kiện liên quan. Metrics thường phản ánh trạng thái tổng hợp của nhiều request nên không xác định trực tiếp request nào gặp vấn đề. Engineer sử dụng `service.name` và time window để tìm các log liên quan, sau đó lấy `trace_id` từ log để mở trace của request cần điều tra.

1.3 Choosing What to Measure

Sau khi hệ thống tạo ra telemetry, vấn đề tiếp theo là xác định metric nào cần được ưu tiên. Một hệ thống có thể tạo ra nhiều metric, nhưng không phải metric nào cũng cần được ưu tiên. Việc thu thập quá nhiều metric mà không có mục đích rõ ràng có thể khiến dashboard trở nên phức tạp và quá trình điều tra mất nhiều thời gian hơn. USE Method, Four Golden Signals và RED Method cung cấp ba cách lựa chọn metric dựa trên đối tượng cần quan sát.



Hình 6: USE kiểm tra tình trạng của resource, Golden Signals đánh giá user-facing service, còn RED theo dõi request tại từng service hoặc endpoint.

USE Method: Resource Condition

Brendan Gregg công bố phương pháp này vào năm 2012 để hỗ trợ performance investigation. Thay vì bắt đầu từ các công cụ hoặc metric có sẵn, engineer liệt kê các resource rồi kiểm tra từng resource theo một checklist thống nhất. Resource là thành phần có năng lực xử lý hữu hạn, chẳng hạn CPU, memory, storage hoặc network interface.

Với mỗi resource, engineer kiểm tra ba nhóm tín hiệu:

- **Utilization:** tỷ lệ thời gian resource bận xử lý công việc.
- **Saturation:** lượng công việc đang phải chờ vì resource chưa thể xử lý kịp.
- **Errors:** số error event xảy ra trên resource.

Ví dụ, với storage, engineer có thể theo dõi tỷ lệ thiết bị đang bận để đánh giá Utilization, độ dài hàng đợi I/O để đánh giá Saturation và số lỗi đọc ghi để kiểm tra Errors.

Four Golden Signals: Overall Service Health

Tình trạng của từng resource chưa phản ánh đầy đủ trải nghiệm của người dùng. Một service vẫn có thể trả response chậm hoặc sai ngay cả khi CPU và memory chưa đạt giới hạn. Google SRE Book đề xuất bốn tín hiệu để theo dõi user-facing service, tức service có hoạt động trực tiếp ảnh hưởng đến trải nghiệm người dùng:

- **Latency:** thời gian cần để xử lý một request.
- **Traffic:** lượng nhu cầu đang được gửi đến service, thường được đo bằng số request mỗi giây.

-
- **Errors:** số lượng hoặc tỷ lệ request không được xử lý đúng.
 - **Saturation:** mức độ service tiến gần giới hạn xử lý.

Latency và Errors phản ánh chất lượng xử lý request hiện tại. Traffic cho biết workload mà service đang tiếp nhận, còn Saturation cho thấy service còn bao nhiêu khả năng để xử lý thêm workload.

RED Method: Request Behavior

Tom Wilkie tạo ra phương pháp này vào năm 2015 để chuẩn hóa cách quan sát microservices và các service xử lý request. RED thường được dùng để instrument service. Instrument nghĩa là bổ sung code hoặc cấu hình để service tạo ra các metric cần thiết.

Với mỗi service hoặc endpoint, RED theo dõi:

- **Rate:** số request được xử lý trong một đơn vị thời gian.
- **Errors:** số lượng hoặc tỷ lệ request thất bại.
- **Duration:** phân bố thời gian xử lý request.

Endpoint là điểm tiếp nhận một loại request cụ thể, chẳng hạn `/checkout` hoặc `/payment`. Khi các service cùng sử dụng Rate, Errors và Duration, dashboard có thể được tổ chức theo một cấu trúc nhất quán và hỗ trợ drill down đến từng endpoint.

Mối quan hệ giữa ba phương pháp

Như minh họa trong Hình 6, ba phương pháp tập trung vào những phạm vi khác nhau và có thể được kết hợp trong cùng một incident. Khi checkout service chậm, Golden Signals cho thấy Latency và Errors đang tăng trong khi Traffic vẫn ổn định. RED giúp thu hẹp vấn đề vào endpoint `/checkout`, nơi Duration tăng và nhiều request thất bại. Nếu dấu hiệu cho thấy vấn đề liên quan đến năng lực xử lý, USE được sử dụng để kiểm tra resource bên dưới và tìm bottleneck.

1.4 Service Reliability with SLI, SLO, and Error Budget

Service reliability là khả năng service duy trì việc cung cấp chức năng đúng theo kỳ vọng của người dùng trong một khoảng thời gian, chẳng hạn xử lý request thành công hoặc phản hồi trong thời gian chấp nhận được. Metric cung cấp các giá trị dùng để quan sát những hành vi này, nhưng bản thân metric chưa cho biết service có đáp ứng mức reliability mong muốn hay không. Để đánh giá rõ ràng hơn, engineer chuyển metric thành SLI, đặt mục tiêu bằng SLO và quy đổi phần sai lệch được chấp nhận thành Error Budget. Trong phần này, cả ba khái niệm được minh họa bằng request success rate của checkout service.



Hình 7: Từ Metric, SLI được tính và so sánh với SLO để xác định Error Budget còn lại.

Chuyển Metric thành SLI

Các metric ban đầu chỉ cung cấp dữ liệu thô, chẳng hạn số request thành công và tổng số request. Từ các giá trị này, engineer xây dựng SLI, viết tắt của Service Level Indicator. Đây là phép đo đại diện cho một khía cạnh cụ thể của service.

Với request success rate, SLI được tính như sau:

$$SLI = \frac{\text{Successful Requests}}{\text{Total Requests}} \times 100\%.$$

Trong đó:

- **Successful Requests** là số request được xử lý thành công.
- **Total Requests** là tổng số request được đưa vào phép đo.

Kết quả SLI cho biết tỷ lệ request mà service xử lý thành công. Giả sử checkout service xử lý 1.000.000 request trong 30 ngày, trong đó 999.200 request thành công:

$$SLI = \frac{999,200}{1,000,000} \times 100\% = 99,92\%.$$

SLI cho biết service đã đạt request success rate 99,92% trong khoảng thời gian được đo.

Đặt mục tiêu bằng SLO

SLI phản ánh kết quả thực tế, nhưng chưa cho biết kết quả đó có đạt yêu cầu hay không. SLO, viết tắt của Service Level Objective, là mục tiêu được đặt trên một SLI trong một khoảng thời gian xác định.

Với checkout service, SLO có thể được đặt như sau:

Trong 30 ngày, ít nhất 99,9% request phải được xử lý thành công.

SLO này gồm ba thành phần:

- **Đối tượng đo:** checkout request.
- **SLI được sử dụng:** request success rate.
- **Mục tiêu và time window:** ít nhất 99,9% trong 30 ngày.

SLI thực tế là 99,92%, cao hơn mục tiêu 99,9%, nên service vẫn đáp ứng SLO trong khoảng thời gian này.

Xác định Error Budget

SLO 99,9% không yêu cầu mọi request đều thành công. Phần sai lệch được SLO chấp nhận được gọi là Error Budget.

Với SLO dựa trên request success rate, Error Budget được tính bằng:

$$\text{Error Budget} = 100\% - \text{SLO}.$$

Khi SLO bằng 99,9%:

$$\text{Error Budget} = 100\% - 99,9\% = 0,1\%.$$

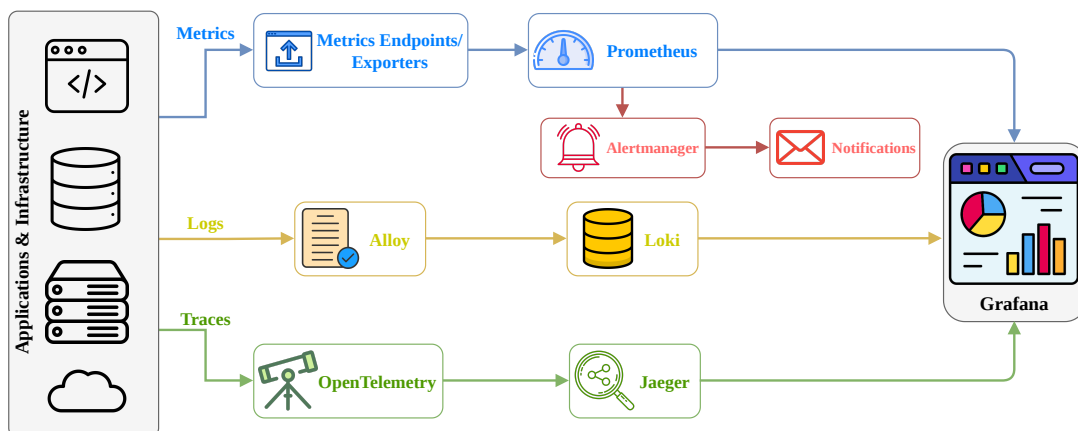
Điều này có nghĩa tối đa 0,1% request được phép thất bại trong 30 ngày. Với tổng cộng 1.000.000 request:

$$1,000,000 \times 0,1\% = 1,000.$$

Checkout service có thể có tối đa 1.000 request thất bại mà vẫn đáp ứng SLO. Trong ví dụ trên, service có 800 request thất bại nên đã sử dụng 80% Error Budget và còn lại 200 request.

1.5 Hands-on Environment Setup

Các phần thực hành về Metrics, Logs và Traces sử dụng chung một observability stack được chuẩn bị sẵn trong repository và triển khai bằng Docker Compose. Environment này chỉ cần thiết lập một lần trước khi tiếp tục với các phần thực hành phía sau.



Hình 8: Kiến trúc observability stack trong tutorial: Prometheus cho Metrics, Alloy và Loki cho Logs, OpenTelemetry và Jaeger cho Traces, còn Grafana dùng để trực quan hóa.

Stack được tổ chức thành ba pipeline chính:

- **Metrics:** Metrics Endpoints / Exporters → Prometheus → Grafana.
- **Logs:** Alloy → Loki → Grafana.
- **Traces:** OpenTelemetry → Jaeger → Grafana.

Clone repository và chuyển vào thư mục project:

```
git clone <REPOSITORY_URL>
cd <REPOSITORY_NAME>
```

Khởi chạy toàn bộ stack:

```
docker compose up -d
```

Kiểm tra trạng thái các service:

```
docker compose ps
```

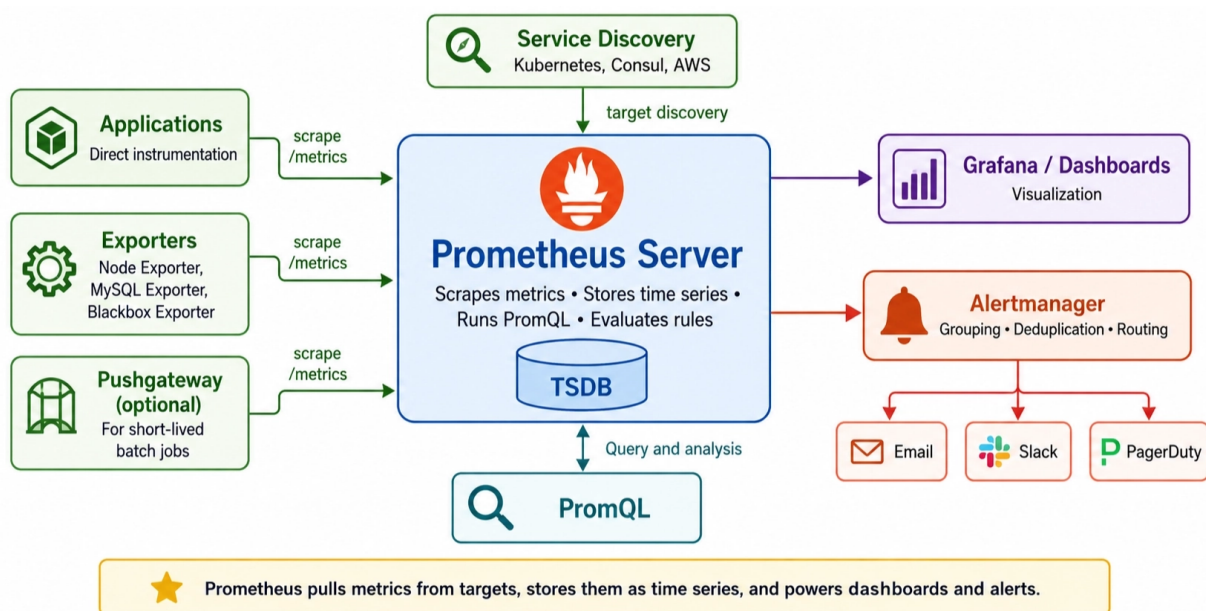
Nếu các service cần thiết đều ở trạng thái **Up** hoặc **running**, environment đã sẵn sàng và có thể được sử dụng cho các phần thực hành tiếp theo.

Các khái niệm trong chương này sẽ được triển khai bằng observability stack đã giới thiệu ở Hình ???. Các chương tiếp theo lần lượt trình bày cách thu thập, truy vấn và trực quan hóa Metrics, Logs và Traces bằng từng nhóm công cụ tương ứng.

2 Metrics Collection and Analysis with Prometheus

2.1 Prometheus and the Metrics Pipeline

Sau khi xác định được các metric cần theo dõi, bước tiếp theo là xây dựng một pipeline có thể thu thập, lưu trữ và truy vấn metrics theo thời gian. Prometheus là hệ thống monitoring chuyên cho dữ liệu time series, trong đó Prometheus Server định kỳ lấy metrics từ các target, lưu chúng vào TSDB và cung cấp dữ liệu cho truy vấn, dashboard và alerting.



Hình 9: Prometheus thu thập metrics từ các target, lưu dữ liệu dưới dạng time series và cung cấp chúng cho truy vấn, trực quan hóa và alerting.

Như minh họa trong Hình 9, Prometheus Server là thành phần trung tâm của pipeline. Các application hoặc exporter đóng vai trò là nguồn metrics, còn TSDB (Time-Series Database) là nơi Prometheus lưu các metric sample theo thời gian. PromQL được sử dụng để truy vấn dữ liệu, trong khi Grafana có thể sử dụng Prometheus làm data source để trực quan hóa các kết quả này.

Trong môi trường nhỏ, danh sách target có thể được cấu hình trực tiếp. Với các môi trường động, Service Discovery có thể hỗ trợ Prometheus tìm và cập nhật target tự động. Prometheus cũng có thể đánh giá alerting rules và chuyển các alert được kích hoạt đến Alertmanager; phần alerting sẽ được trình bày chi tiết ở chương sau.

Pull-based Metrics Collection

Prometheus chủ yếu sử dụng cơ chế pull-based collection, nghĩa là Prometheus chủ động kết nối đến các target để lấy metrics theo một chu kỳ xác định.

Giả sử một checkout service đã cung cấp endpoint metrics tại:

```
http://checkout-service:8000/metrics
```

Prometheus sẽ định kỳ gửi HTTP request đến endpoint này để thu thập các metric sample hiện tại. Mỗi lần Prometheus thực hiện quá trình thu thập từ một target được gọi là một *scrape*.

Trong phần này, `/metrics` chỉ được xem là endpoint mà Prometheus sử dụng để lấy dữ liệu. Cách application hoặc exporter tạo ra nội dung tại endpoint này sẽ được trình bày ở Mục 2.2.

Targets, `scrape_interval` và trạng thái scrape

Để thu thập metrics, Prometheus trước hết cần biết những endpoint nào phải được theo dõi. Mỗi endpoint mà Prometheus định kỳ truy cập để lấy metrics được gọi là một target.

Trong một cấu hình đơn giản, các target có thể được khai báo trực tiếp trong `prometheus.yml`. Ví dụ:

```
scrape_configs:
- job_name: "checkout-service"
  static_configs:
  - targets:
    - "checkout-service:8000"
```

Ở đây, `checkout-service:8000` là target mà Prometheus sẽ scrape. Trường `job_name` đặt tên cho nhóm target này, giúp các metrics được thu thập được nhận diện theo cùng một vai trò.

Sau khi biết cần scrape target nào, Prometheus cần xác định tần suất thu thập dữ liệu. Khoảng thời gian giữa hai lần scrape được cấu hình bằng `scrape_interval`. Ví dụ:

```
global:
  scrape_interval: 15s
```

Với cấu hình trên, Prometheus sẽ cố gắng lấy metrics từ các target khoảng 15 giây một lần. Mỗi lần scrape tạo ra một tập metric sample mới với timestamp tương ứng. Khi quá trình này lặp lại, các sample được lưu theo thời gian và tạo thành dữ liệu time series.

Việc chọn `scrape_interval` là sự cân bằng giữa độ chi tiết và lượng dữ liệu cần xử lý. Khoảng thời gian ngắn giúp ghi nhận thay đổi nhanh hơn nhưng tạo ra nhiều sample hơn; khoảng thời gian dài làm giảm lượng dữ liệu nhưng có thể không phản ánh được những biến động ngắn.

Khi một target được scrape, Prometheus thường bổ sung các label để xác định nguồn dữ liệu. Hai label thường gặp là `job`, đại diện cho nhóm target, và `instance`, đại diện cho target cụ thể. Với cấu hình trên, một metric có thể mang các label:

```
job="checkout-service"
instance="checkout-service:8000"
```

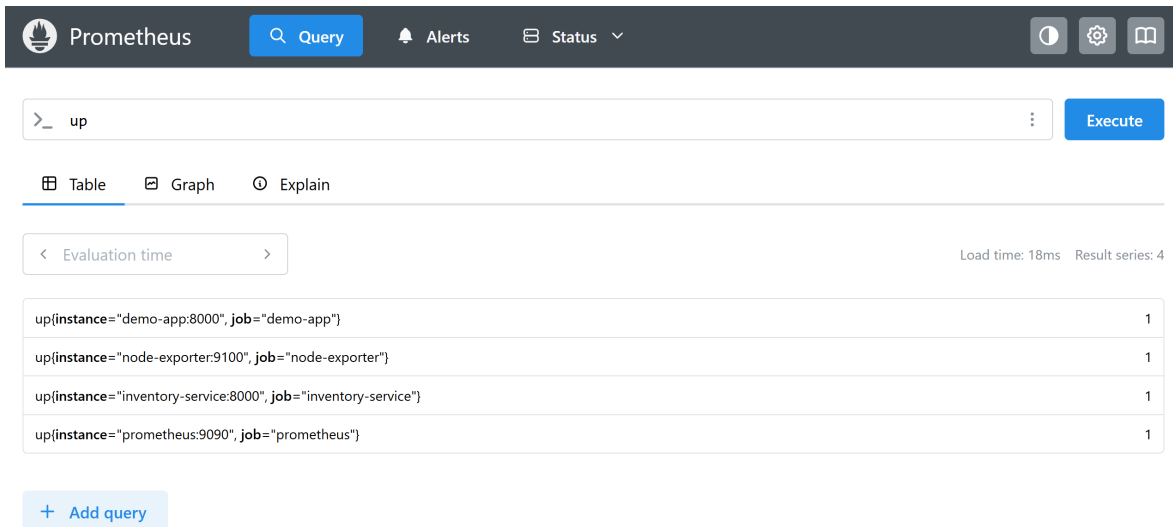
Các label này giúp engineer phân biệt dữ liệu đến từ service hoặc instance nào khi truy vấn. Cách Prometheus sử dụng labels để xác định time series sẽ được trình bày kỹ hơn ở Mục 2.3.

Prometheus đồng thời tự tạo metric `up` cho mỗi target để phản ánh kết quả của lần scrape gần nhất. Ví dụ:

```
up{job="demo-app", instance="demo-app:8000"}
```

Kết quả 1 cho biết Prometheus đã thu thập metrics từ target thành công, trong khi 0 cho biết lần scrape gần nhất thất bại. Nhờ đó, `up` cung cấp một cách đơn giản để kiểm tra liệu Prometheus còn có thể tiếp cận và thu thập dữ liệu từ target hay không.

Tuy nhiên, metric này chỉ phản ánh scrape status, không phản ánh đầy đủ application health. Một checkout service vẫn có thể trả về endpoint metrics bình thường trong khi database hoặc một dependency phía sau đang gặp lỗi. Vì vậy, `up` thường được dùng như bước kiểm tra đầu tiên trước khi engineer xem xét thêm các tín hiệu như latency, error rate hoặc resource usage.



Hình 10: Trạng thái scrape của các target trong Prometheus.

Service Discovery

Khai báo target thủ công phù hợp với lab hoặc hệ thống có ít thành phần. Trong môi trường động, instance có thể được tạo mới, thay đổi địa chỉ hoặc bị loại bỏ thường xuyên, khiến danh sách target khó được duy trì bằng cấu hình tĩnh.

Service Discovery cho phép Prometheus lấy danh sách target từ các nền tảng như Kubernetes, Consul hoặc cloud environment và cập nhật chúng khi hạ tầng thay đổi. Ở mức overview, có thể hiểu Service Discovery là lớp giúp Prometheus xác định cần scrape những target nào, còn quá trình scrape metrics vẫn được Prometheus thực hiện theo cơ chế pull.

Short-lived Jobs và Pushgateway

Pull-based collection phù hợp với các service tồn tại đủ lâu để Prometheus có thể scrape theo chu kỳ. Tuy nhiên, một short-lived job có thể hoàn thành trước khi Prometheus thực hiện lần scrape tiếp theo.

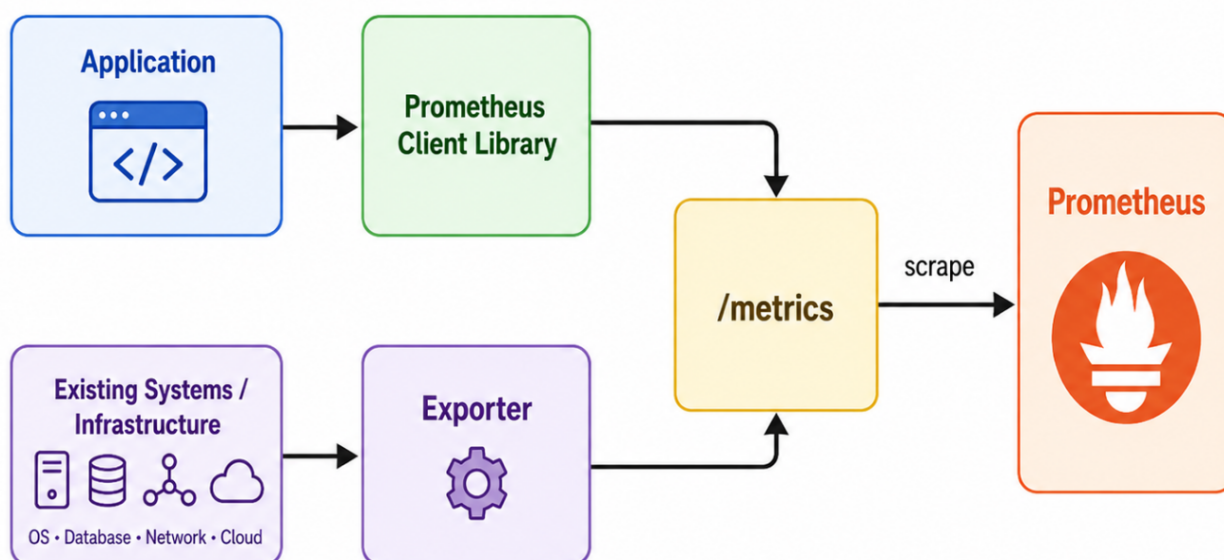
Ví dụ, nếu một batch job chỉ chạy trong 5 giây trong khi `scrape_interval` là 15 giây, Prometheus có thể không kịp thu thập metrics trực tiếp từ job đó. Trong trường hợp này, job có thể gửi metrics đến Pushgateway trước khi kết thúc, sau đó Prometheus scrape Pushgateway như một target thông thường.

Pushgateway vì vậy chỉ đóng vai trò trung gian cho một số workload tồn tại trong thời gian ngắn. Cơ chế thu thập chính của Prometheus vẫn là pull-based collection.

2.2 Exposing Metrics: Application Instrumentation and Exporters

Ở Mục 2.1, Prometheus được xem như thành phần chủ động scrape metrics từ các target thông qua endpoint `/metrics`. Câu hỏi tiếp theo là: những metrics xuất hiện tại endpoint đó được tạo ra từ đâu?

Trong thực tế, có hai cách phổ biến để cung cấp metrics cho Prometheus. Application có thể tự instrument để tạo metrics từ bên trong source code, hoặc sử dụng exporter để chuyển dữ liệu từ những hệ thống có sẵn sang định dạng mà Prometheus có thể thu thập.



Hình 11: Application có thể tự instrument để expose metrics, trong khi exporter chuyển dữ liệu từ các hệ thống có sẵn sang định dạng mà Prometheus có thể thu thập.

Application Instrumentation

Instrumentation là quá trình bổ sung code hoặc cấu hình để application tạo ra các telemetry cần thiết trong quá trình hoạt động. Với Prometheus, application thường sử dụng Prometheus

client library để định nghĩa, cập nhật và expose metrics.

Cách này phù hợp với những tín hiệu mà chính application hiểu rõ nhất, chẳng hạn:

- tổng số request đã xử lý.
- số request đang được xử lý.
- thời gian xử lý request.
- số request thất bại.
- business metrics như số order hoặc transaction.

Ví dụ, khi một HTTP request được xử lý, application có thể tăng request counter và ghi nhận thời gian xử lý. Client library duy trì các giá trị này và expose chúng qua endpoint `/metrics` để Prometheus scrape.

Application instrumentation vì vậy phù hợp khi team kiểm soát source code và cần theo dõi application-level hoặc business-level behavior, thay vì chỉ quan sát tài nguyên hạ tầng bên dưới.

Exporters

Không phải hệ thống nào cũng có thể được instrument trực tiếp bằng Prometheus client library. Operating system, database, network device hoặc phần mềm có sẵn thường cung cấp dữ liệu thông qua system files, command, API hoặc giao thức riêng.

Trong trường hợp này, một exporter đóng vai trò trung gian. Exporter đọc dữ liệu từ hệ thống gốc, chuyển các giá trị cần thiết thành Prometheus metrics và expose chúng qua endpoint để Prometheus scrape.

Một ví dụ phổ biến là Node Exporter, được sử dụng để cung cấp system-level metrics từ Linux và Unix. Node Exporter đọc dữ liệu do operating system cung cấp và chuyển chúng thành các metrics về CPU, memory, disk, filesystem và network.

Một số metric thường gặp gồm:

- `node_cpu_seconds_total`: tổng thời gian CPU đã sử dụng theo từng core và mode.
- `node_memory_MemAvailable_bytes`: lượng memory còn khả dụng.
- `node_disk_read_bytes_total`: tổng số byte đã được đọc từ disk.
- `node_network_receive_bytes_total`: tổng số byte đã nhận qua network interface.
- `node_filesystem_avail_bytes`: dung lượng filesystem còn khả dụng.

Prefix `node_` giúp nhận biết metric được cung cấp bởi Node Exporter. Như vậy, application instrumentation phù hợp với application hoặc business metrics, còn exporter phù hợp khi cần lấy metrics từ những hệ thống mà team không muốn hoặc không thể sửa source code.

Prometheus Exposition Format

Dù metrics được tạo trực tiếp bởi application hay thông qua exporter, endpoint mà Prometheus scrape cần cung cấp dữ liệu theo một format mà Prometheus có thể parse.

Ví dụ:

```
# HELP http_requests_total Total number of HTTP requests.
# TYPE http_requests_total counter
http_requests_total{service="checkout",status="200"} 15230
```

Trong đó:

- # HELP mô tả ý nghĩa của metric;
- # TYPE khai báo loại metric;
- dòng cuối chứa metric sample thực tế.

Một metric sample thường có cấu trúc `metric_name{labels} value`. Trong đó, labels là các cặp key-value đóng vai trò metadata, giúp bổ sung context để biết metric đang mô tả service, status hoặc nhóm tài nguyên nào. Ví dụ:

```
http_requests_total{service="checkout", status="200"} 15230
```

Ở đây, `http_requests_total` là tên metric, `service="checkout"` cho biết metric thuộc service checkout, `status="200"` cho biết các request có HTTP status 200, còn 15230 là giá trị hiện tại. Khi Prometheus scrape endpoint này, sample được lưu cùng timestamp để tạo dữ liệu theo thời gian. Cách labels xác định một time series và ảnh hưởng đến cardinality sẽ được trình bày kỹ hơn ở Mục 2.3.

```
python_gc_collections_total{generations="2"} 1.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patchlevel="14",version="3.12.14"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 9.32868096e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 8.553152e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.78704570069e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 3.69
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 23.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
app_requests_total{endpoint="/metrics/",method="GET",status="200"} 37.0
app_requests_total{endpoint="/checkout",method="GET",status="200"} 10.0
app_requests_total{endpoint="/checkout",method="GET",status="502"} 2.0
# HELP app_requests_created Total number of HTTP requests
# TYPE app_requests_created gauge
app_requests_created{endpoint="/metrics/",method="GET",status="200"} 1.7870457049443667e+09
app_requests_created{endpoint="/checkout",method="GET",status="200"} 1.7870457059807475e+09
app_requests_created{endpoint="/checkout",method="GET",status="502"} 1.787045723440937e+09
```

```
45 REQUEST_COUNT = Counter(
46     "app_requests_total",
47     "Total number of HTTP requests",
48     ["method", "endpoint", "status"],
49 )
50
51 REQUEST_LATENCY = Histogram(
52     "app_request_latency_seconds",
53     "HTTP request latency in seconds",
54     ["method", "endpoint"],
55 )
56
57 app = FastAPI()
58
59
60 @app.middleware("http")
61 async def collect_metrics(request, call_next):
62     if request.url.path == "/metrics":
63         return await call_next(request)
64
65     start = time.time()
66     response = await call_next(request)
67     route = request.scope.get("route")
68     endpoint = route.path if route else request.url.path
69
70     REQUEST_LATENCY.labels(request.method, endpoint).observe(time.time() - start)
71     REQUEST_COUNT.labels(request.method, endpoint, str(response.status_code)).inc()
72     log_event(
73         "info" if response.status_code < 500 else "error",
74         "http_request",
75         method=request.method,
76         path=request.url.path,
77         endpoint=endpoint,
78         status_code=response.status_code,
79         duration_ms=round((time.time() - start) * 1000, 2),
80     )
81
82     return response
```

Hình 12: Endpoint `/metrics` cung cấp metadata và các metric sample theo Prometheus exposition format.

Metric Naming Convention

Tên metric nên mô tả rõ đại lượng đang được đo và sử dụng đơn vị nhất quán. Một số quy ước thường gặp:

- **_total**: thường dùng cho Counter. Ví dụ, `http_requests_total` biểu diễn tổng số HTTP request đã được xử lý.
- **_seconds**: dùng cho các đại lượng thời gian tính theo giây. Ví dụ, `http_request_duration_seconds` biểu diễn thời gian xử lý HTTP request.
- **_bytes**: dùng cho các đại lượng dung lượng tính theo byte. Ví dụ, `node_memory_MemAvailable_bytes` biểu diễn lượng memory còn khả dụng.

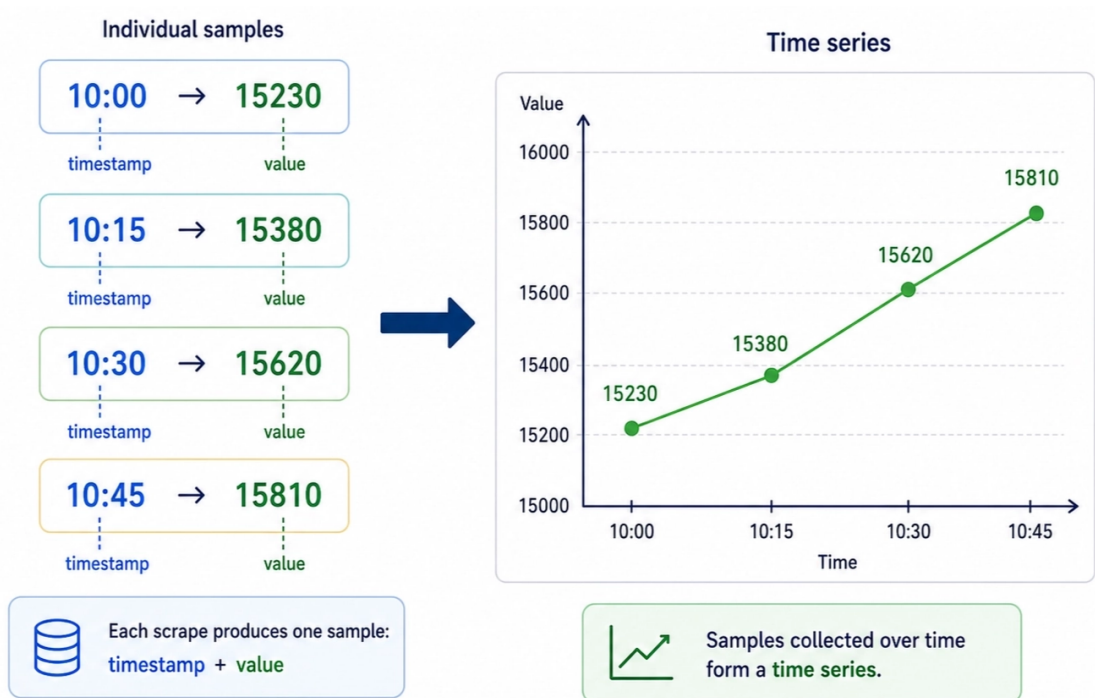
Việc đặt tên nhất quán giúp engineer nhận biết nhanh ý nghĩa và đơn vị của metric, đồng thời làm PromQL và dashboard dễ đọc hơn.

2.3 Prometheus Time-Series Data Model

Sau khi Prometheus scrape metrics từ các target, dữ liệu không chỉ được giữ dưới dạng giá trị hiện tại mà được lưu liên tục theo thời gian. Cách tổ chức này cho phép Prometheus theo dõi sự thay đổi của một metric, so sánh các thời điểm khác nhau và thực hiện các phép tính như rate, trend hoặc percentile ở bước truy vấn sau này.

Metric Samples và Time Series

Mỗi lần Prometheus scrape một target, nó thu được các metric sample. Mỗi sample ghi lại value của metric tại một timestamp cụ thể. Khi cùng một metric tiếp tục được scrape ở các thời điểm khác nhau, các sample này được lưu nối tiếp nhau và hình thành một time series.



Hình 13: Mỗi lần scrape tạo ra một sample gồm timestamp và value. Các sample được thu thập liên tục theo thời gian sẽ hình thành một time series.

Việc lưu lại toàn bộ chuỗi sample thay vì chỉ giữ giá trị mới nhất cho phép Prometheus phân tích sự thay đổi của metric theo thời gian. Để xác định những sample nào thuộc cùng một time series, Prometheus dựa trên **metric name** và toàn bộ **label set** đi kèm.

Tuy nhiên, để biết những sample nào thuộc cùng một time series, Prometheus không chỉ dựa vào tên metric. Một time series được xác định bởi metric name cùng toàn bộ label set đi kèm.

Ví dụ, `http_requests_total{method="GET",status="200"}`. Nếu một label thay đổi, Prometheus xem đó là một time series khác. Vì vậy:

```
http_requests_total{method="GET",status="200"}
http_requests_total{method="POST",status="200"}
http_requests_total{method="POST",status="500"}
```

là ba time series riêng dù đều sử dụng cùng metric name `http_requests_total`.

Labels

Labels là các cặp **key=value** dùng để mô tả thêm các chiều của metric. Nhờ labels, một metric có thể được phân tích theo service, endpoint, status code, instance hoặc environment mà không cần tạo một metric name riêng cho từng trường hợp.

Ví dụ:

```
http_requests_total{
  service="checkout",
  endpoint="/payment",
  status="500"
}
```

Trong metric trên, các labels giúp mô tả metric theo nhiều chiều khác nhau. Nhờ đó, Prometheus có thể phân biệt dữ liệu theo service, endpoint hoặc trạng thái request và lựa chọn đúng time series khi truy vấn.

- `service="checkout"`: request thuộc checkout service.
- `endpoint="/payment"`: request được xử lý tại endpoint `/payment`.
- `status="500"`: request kết thúc với server error.
- `job="checkout-service"`: nhóm các target có cùng vai trò.
- `instance="checkout-service:8000"`: xác định target cụ thể mà Prometheus đã scrape.

Trong đó, `service`, `endpoint` và `status` thường đến từ application hoặc exporter, còn `job` và `instance` thường được Prometheus bổ sung khi scrape. Nhờ cách tổ chức này, cùng một service có thể chạy trên nhiều instance mà dữ liệu vẫn được phân biệt rõ ràng.

Labels sau này cũng trở thành cơ sở để filter dữ liệu bằng PromQL, chẳng hạn chọn riêng một service, một endpoint, các request có `status="500"` hoặc một instance đang cần điều tra. Phần hands-on ở Mục 2.4 sẽ sử dụng trực tiếp các label này trong truy vấn.

Cardinality

Việc thêm labels giúp metric trở nên linh hoạt hơn, nhưng mỗi tổ hợp label values khác nhau cũng tạo thêm một time series mới. Số lượng time series được tạo ra từ các tổ hợp này được gọi là cardinality.

Ví dụ, nếu một metric được phân chia theo 20 endpoint, 5 status code, 10 instance và 3 environment thì số tổ hợp có thể đạt:

$$20 \times 5 \times 10 \times 3 = 3,000 \text{ time series.}$$

Điều này cho thấy cardinality có thể tăng rất nhanh ngay cả khi mỗi label riêng lẻ chỉ có một số lượng giá trị vừa phải.

Rủi ro lớn hơn xuất hiện khi sử dụng những giá trị gần như luôn khác nhau làm label, chẳng hạn `user_id`, `request_id`, email hoặc timestamp. Nếu `request_id` được dùng làm label và mỗi request có một ID riêng, gần như mỗi request sẽ tạo thêm một time series mới. Khi số lượng series tăng quá lớn, Prometheus phải sử dụng nhiều memory và storage hơn, đồng thời query cũng trở nên tốn kém hơn.

Vì vậy, labels nên tập trung vào những chiều dữ liệu có số lượng giá trị tương đối ổn định và thực sự hữu ích cho monitoring, chẳng hạn service, status, endpoint, environment hoặc instance.

2.4 Hands-on: From Raw Metrics to Operational Signals with PromQL

Các ví dụ trong phần này sử dụng cùng một monitoring stack gồm checkout application, Node Exporter và Prometheus. Các operational signals được xây dựng và kiểm tra trực tiếp trên Prometheus trước khi được sử dụng lại để tạo Grafana dashboard ở Mục 2.5.

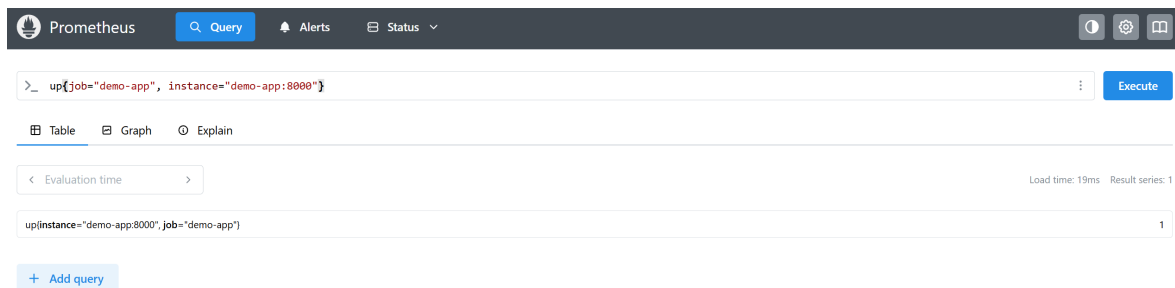
Các time series được lưu trong Prometheus mới chỉ là dữ liệu thô. Để trả lời những câu hỏi vận hành như service có đang hoạt động không, traffic hiện tại bao nhiêu, tỷ lệ lỗi có tăng hay CPU và memory có gần quá tải không, các time series này cần được lựa chọn và tính toán thành những tín hiệu cụ thể.

PromQL (Prometheus Query Language) là ngôn ngữ truy vấn của Prometheus dùng để thực hiện quá trình đó. PromQL cho phép chọn metric, lọc theo labels, lấy dữ liệu trong một khoảng thời gian, tính tốc độ thay đổi và tổng hợp nhiều time series. Cách truy vấn cũng phụ thuộc vào metric type, vì Counter, Gauge và Histogram biểu diễn dữ liệu theo những cách khác nhau. Phần này sử dụng các bài toán monitoring thực tế để lần lượt tạo ra những operational signals như Target Health, Request Rate, Error Rate, CPU Usage, Memory Usage, P95 Latency và Availability SLI.

Target Health

Metric `up` cho biết Prometheus có scrape một target thành công hay không. Có thể query trực tiếp toàn bộ target hoặc kết hợp với labels như `job` và `instance` để kiểm tra một target cụ thể.

```
up{job="demo-app", instance="demo-app:8000"}
```



Hình 14: Truy vấn trạng thái scrape của checkout service bằng metric `up`.

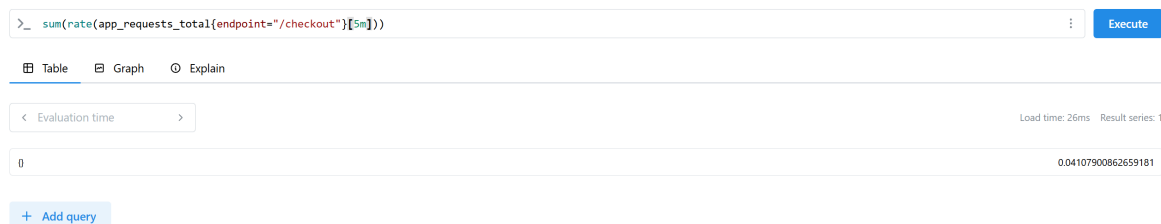
Giá trị 1 cho biết lần scrape gần nhất thành công, còn 0 cho biết Prometheus không thể thu thập metrics từ target đó.

Counter to Request Rate

`app_requests_total` là Counter, biểu diễn tổng số HTTP request mà demo-app đã xử lý. Mỗi request hoàn tất làm Counter tăng lên, với các labels `method`, `endpoint` và `status`. Giá trị hiện tại của Counter chỉ cho biết tổng số request đã xảy ra, chưa phản ánh traffic tại thời điểm quan sát.

Để tính Request Rate, PromQL sử dụng `rate()` trên dữ liệu của 5 phút gần đây và có thể aggregate nhiều time series bằng `sum()`. Kết quả biểu diễn số request trung bình mỗi giây, tương ứng với tín hiệu Traffic.

```
sum(rate(app_requests_total{endpoint="/checkout"}[5m]))
```



Hình 15: Request Rate trung bình của demo-app đối với checkout request trong cửa sổ 5 phút.

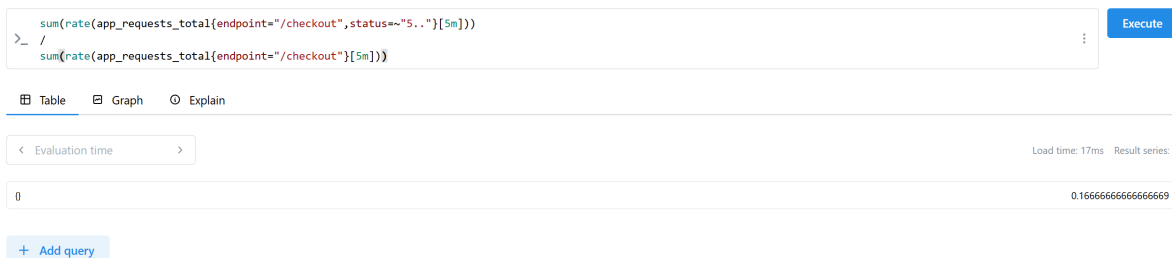
Kết quả 0.04 trong hình có thể hiểu demo-app xử lý trung bình khoảng 0.04 checkout request mỗi giây trong 5 phút gần đây. Nếu chưa có dữ liệu, chạy `docker compose up -d seed-traffic`, đợi khoảng 20 giây rồi query lại.

Counter to Error Rate

Cùng một `http_requests_total` có thể được sử dụng để theo dõi lỗi thông qua label `status`. PromQL lọc các request có HTTP status thuộc nhóm 5xx, tính tốc độ phát sinh lỗi rồi so sánh với tổng request rate để thu được Error Rate.

Trong bước này, người học đồng thời làm quen với label filtering, regular expression matcher và phép tính ratio trong PromQL.

```
sum(rate(app_requests_total{endpoint="/checkout",status=~"5.."}[5m]))
/
sum(rate(app_requests_total{endpoint="/checkout"}[5m]))
```



Hình 16: Tỷ lệ request kết thúc bằng HTTP status 5xx trong cửa sổ 5 phút.

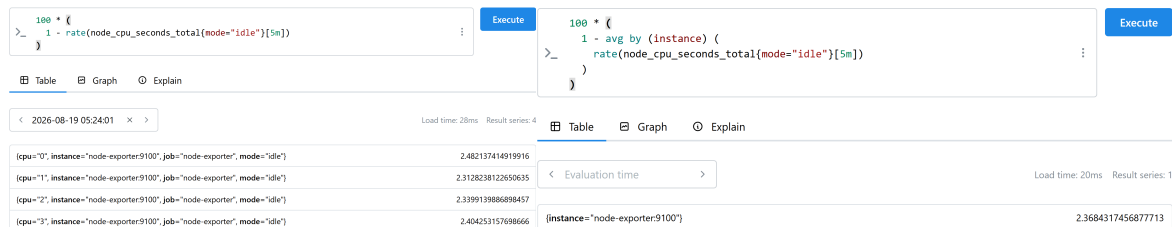
Kết quả 0.1667 cho biết khoảng 16.67% checkout request trong 5 phút gần đây kết thúc bằng server error. Với traffic hiện tại, tỷ lệ này tương ứng 2 request lỗi trên tổng 12 checkout request. Đây là tín hiệu Errors trong Four Golden Signals.

Counter to CPU Usage

Một ví dụ khác của Counter là `node_cpu_seconds_total`, metric ghi nhận tổng thời gian CPU đã dành cho từng mode như `idle`, `user`, `system` hoặc `iowait`. Do CPU time được cộng dồn theo thời gian, metric này tiếp tục được xử lý bằng `rate()` để suy ra mức sử dụng CPU.

Để tính CPU usage, PromQL thường xác định tốc độ tăng của CPU time ở `mode="idle"`, sau đó lấy phần còn lại làm tỷ lệ CPU đang được sử dụng. Khi host có nhiều CPU core, các giá trị cũng cần được aggregate theo instance.

```
100 * (
  1 - avg by (instance) (
    rate(node_cpu_seconds_total{mode="idle"}[5m])
  )
)
```



Hình 17: CPU usage theo từng core (trái) và CPU usage trung bình theo instance sau khi aggregate (phải).

Trong hình, từng core có CPU usage khoảng 2.3–2.5%, còn `avg by (instance)` gộp bốn core thành mức trung bình khoảng 2.4% cho `node-exporter:9100`. `node_cpu_seconds_total` vẫn là Counter vì metric này ghi nhận CPU time tích lũy.

Gauge to Memory Usage

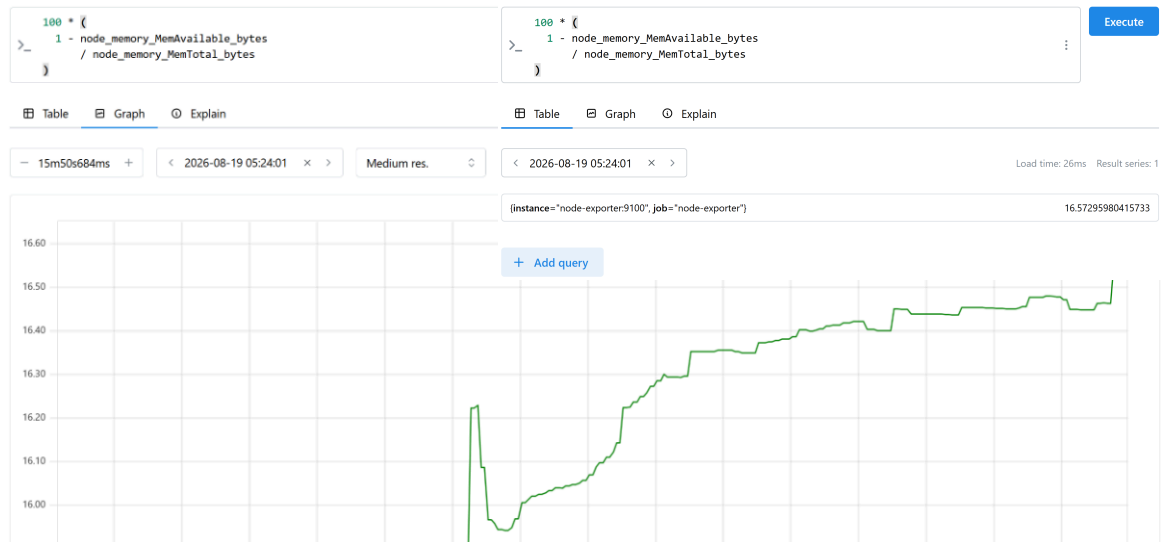
Sau Counter, một metric type phổ biến khác là Gauge. Nếu Counter dùng để biểu diễn một giá trị tích lũy chỉ tăng theo thời gian, thì Gauge biểu diễn trạng thái hiện tại của một giá trị tại thời điểm được quan sát và có thể tăng hoặc giảm giữa các lần scrape..

Node Exporter cung cấp hai Gauge thường dùng để quan sát memory:

- `node_memory_MemTotal_bytes`: tổng lượng memory của host.
- `node_memory_MemAvailable_bytes`: lượng memory còn khả dụng.

Từ hai giá trị này, PromQL có thể tính tỷ lệ memory đang được sử dụng bằng phép toán trực tiếp. Khác với Counter, Gauge không cần dùng `rate()` vì giá trị của nó đã phản ánh trạng thái tại thời điểm scrape.

```
100 * (
  1 - node_memory_MemAvailable_bytes
    / node_memory_MemTotal_bytes
)
```



Hình 18: Tỷ lệ memory đang được sử dụng trên từng host.

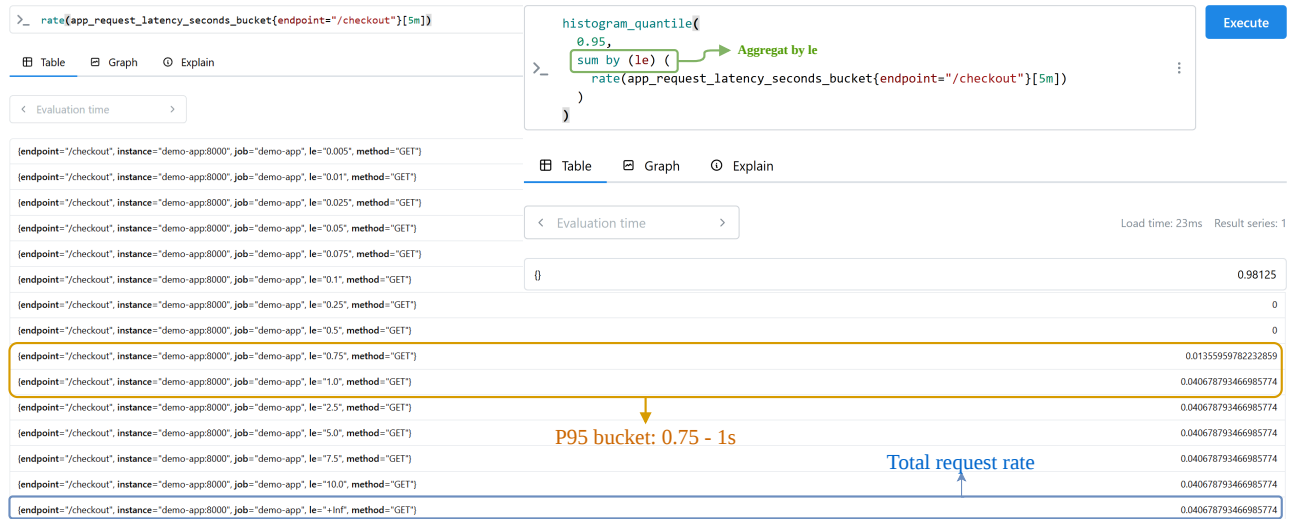
Kết quả trong hình dao động quanh 16%, cho biết môi trường mà Node Exporter quan sát đang sử dụng khoảng 16% memory. Đây là một tín hiệu thường được dùng để quan sát Saturation.

Histogram to P95 Latency

Mỗi request có thể có thời gian xử lý khác nhau, vì vậy latency cần được quan sát trên nhiều request thay vì chỉ một giá trị riêng lẻ. Prometheus thường sử dụng Histogram để ghi nhận phân phối này bằng cách nhóm các observation vào các bucket theo những ngưỡng xác định trước.

Các metric `_bucket` sử dụng label `le` (*less than or equal*) để cho biết có bao nhiêu observation nhỏ hơn hoặc bằng từng ngưỡng. Một observation thuộc bucket nhỏ cũng được tính vào các bucket có ngưỡng lớn hơn. Histogram còn cung cấp `_count` là tổng số observation và `_sum` là tổng giá trị đã ghi nhận. Từ các bucket này, PromQL có thể dùng `histogram_quantile()` để ước tính percentile như p95 latency.

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(app_request_latency_seconds_bucket{
      endpoint="/checkout"
    }[5m])
  )
)
```



Hình 19: P95 request latency của checkout service trong cửa sổ 5 phút.

Trong kết quả, bucket `le="+Inf"` cho biết tổng request rate khoảng 0.04068 req/s. Mốc P95 tương ứng với 95% tổng rate:

$$q = 0.95 \times 0.04068 \approx 0.03864.$$

Giá trị này nằm giữa bucket `le="0.75"` với rate khoảng 0.01356 req/s và bucket `le="1.0"` với rate khoảng 0.04068 req/s. Vì vậy, P95 nằm trong khoảng 0.75–1.0 giây.

Với một quantile nằm giữa hai bucket, phép nội suy có thể được biểu diễn bằng công thức:

$$\text{Quantile} = L + \frac{q - C_{\text{prev}}}{C_{\text{upper}} - C_{\text{prev}}}(U - L).$$

Trong đó, L và U là cận dưới và cận trên của bucket chứa quantile, C_{prev} là cumulative rate trước bucket đó, C_{upper} là cumulative rate tại cận trên, còn q là vị trí quantile cần tìm.

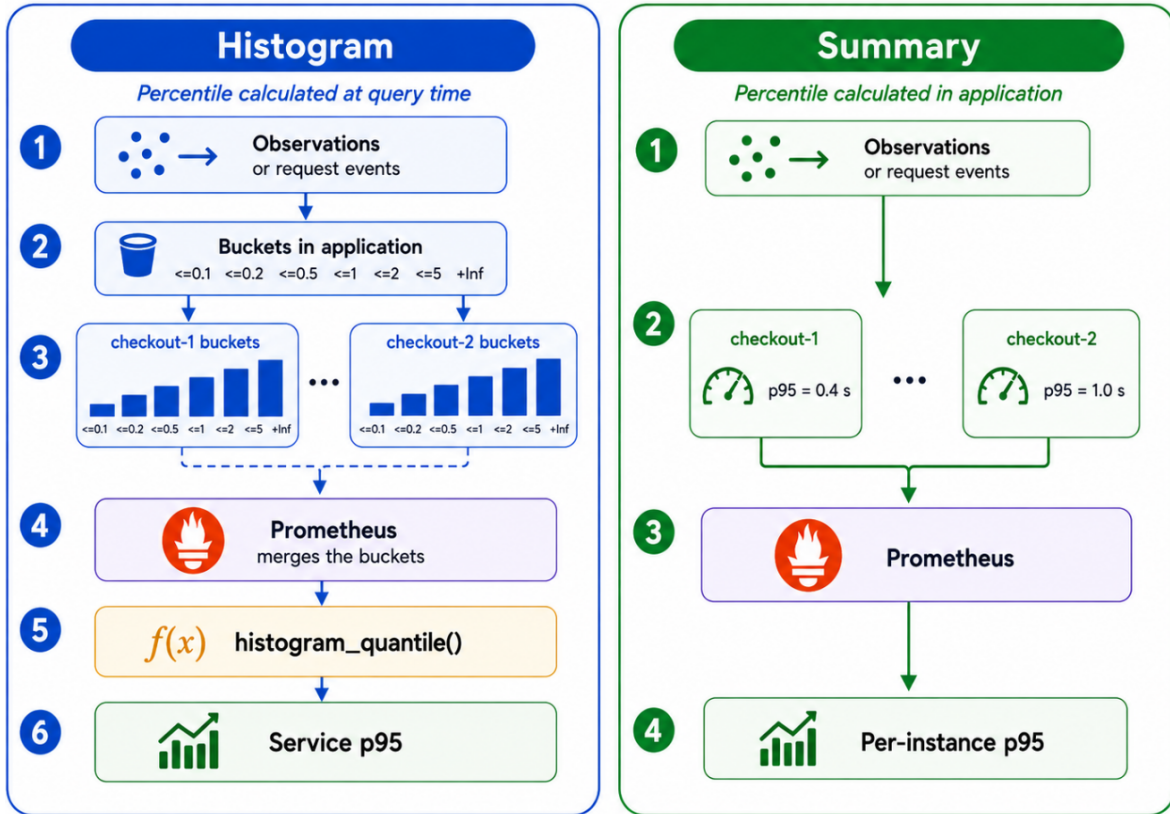
Thay các giá trị từ kết quả:

$$\text{P95} = 0.75 + \frac{0.03864 - 0.01356}{0.04068 - 0.01356}(1.0 - 0.75) \approx 0.98125 \text{ s}.$$

Có thể hiểu rằng khoảng 95% request hoàn thành trong không quá 0.981 giây trong cửa sổ 5 phút. Đây là tín hiệu Latency trong Four Golden Signals.

Histogram và Summary

Histogram và Summary đều được dùng để theo dõi phân phối dữ liệu như request latency, nhưng khác nhau ở cách percentile được tính. Với Histogram, application ghi observations vào các bucket và Prometheus sử dụng dữ liệu đó để tính percentile tại thời điểm query. Với Summary, percentile như p95 được tính trực tiếp tại application trước khi metrics được expose.



Hình 20: Histogram giữ phân phối dưới dạng bucket để Prometheus tính percentile khi query, trong khi Summary tính quantile ngay tại application trước khi expose metrics.

Sự khác biệt này ảnh hưởng trực tiếp đến khả năng tính percentile cho toàn service. Với Histogram, các bucket từ nhiều instance có thể được gộp lại trước, sau đó Prometheus mới tính p95 trên toàn bộ request. Summary thì tạo p95 riêng tại từng instance. Ví dụ:

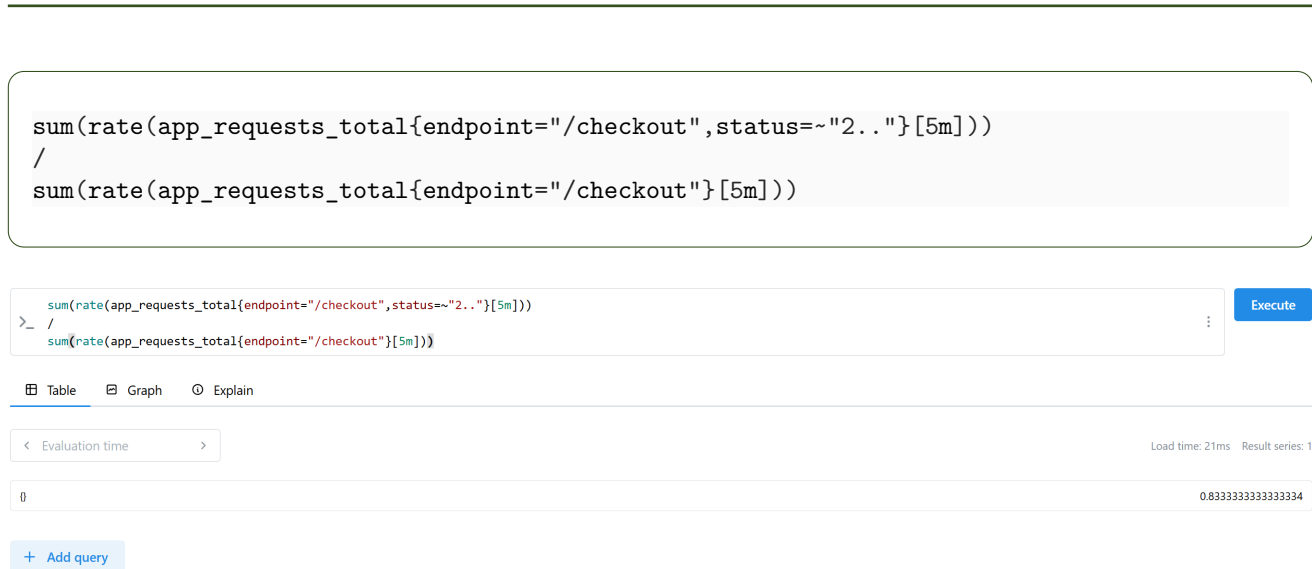
- checkout-1: 1,000 requests, p95 = 0.4 s.
- checkout-2: 20 requests, p95 = 1.0 s.

Nếu lấy trung bình hai giá trị này sẽ được 0.7 giây, nhưng kết quả đó xem hai instance như có ảnh hưởng ngang nhau dù phần lớn request thực tế được xử lý bởi **checkout-1**. Vì vậy, p95 của toàn service phải được tính từ phân phối của tất cả request, không thể suy ra bằng cách lấy trung bình các p95 riêng lẻ.

Availability SLI

Metrics có thể được kết hợp để tính SLI (Service Level Indicator) đã giới thiệu ở Chương 1. Với một HTTP service, Availability SLI có thể được xác định bằng tỷ lệ giữa số request thành công và tổng số request trong cùng một khoảng thời gian.

PromQL trước hết tính rate của successful requests và total requests, sau đó lấy tỷ lệ giữa hai giá trị để thu được Availability SLI.



Hình 21: Availability SLI được tính từ tỷ lệ successful requests trên total requests.

Kết quả 0.8333 cho biết Availability SLI của checkout service trong 5 phút gần đây là khoảng 83.33%. Trong khoảng này, 10 trong tổng 12 request kết thúc thành công với HTTP status 2xx. Giá trị này có thể được so sánh với SLO để đánh giá mức độ đáp ứng reliability và phần Error Budget đã sử dụng.

Operational Signals Overview

Sau các truy vấn trên, Bảng 1 tóm tắt cách PromQL chuyển raw metrics thành các operational signals. Mỗi signal giúp quan sát một khía cạnh khác nhau của sức khỏe và hiệu năng service.

Operational Signal	Raw Metric / Type	Cách xử lý	Ý nghĩa
Target Health	up	Chọn metric và lọc theo labels	Kiểm tra Prometheus có scrape được target hay không
Request Rate	Counter	Tính tốc độ tăng và aggregate	Mức traffic hiện tại của service
Error Rate	Counter	Lọc request lỗi, tính rate và tỷ lệ	Tỷ lệ request thất bại
Memory Usage	Gauge	Tính toán trực tiếp từ các giá trị hiện tại	Mức memory đang được sử dụng
CPU Usage	Counter	Tính rate của CPU time và aggregate	Mức CPU đang được sử dụng
P95 Latency	Histogram	Tính percentile từ histogram buckets	Mức latency mà khoảng 95% request không vượt quá
Availability SLI	Counter	Tính tỷ lệ successful requests trên total requests	Tỷ lệ request thành công

Bảng 1: Từ raw metrics đến các operational signals bằng PromQL.

Các query trên đã tạo ra tập operational signals cần thiết cho việc theo dõi checkout service. Thay vì tiếp tục quan sát từng kết quả riêng lẻ trong Prometheus Query UI, bước tiếp theo là sử dụng chính các query này để xây dựng một dashboard trên Grafana.

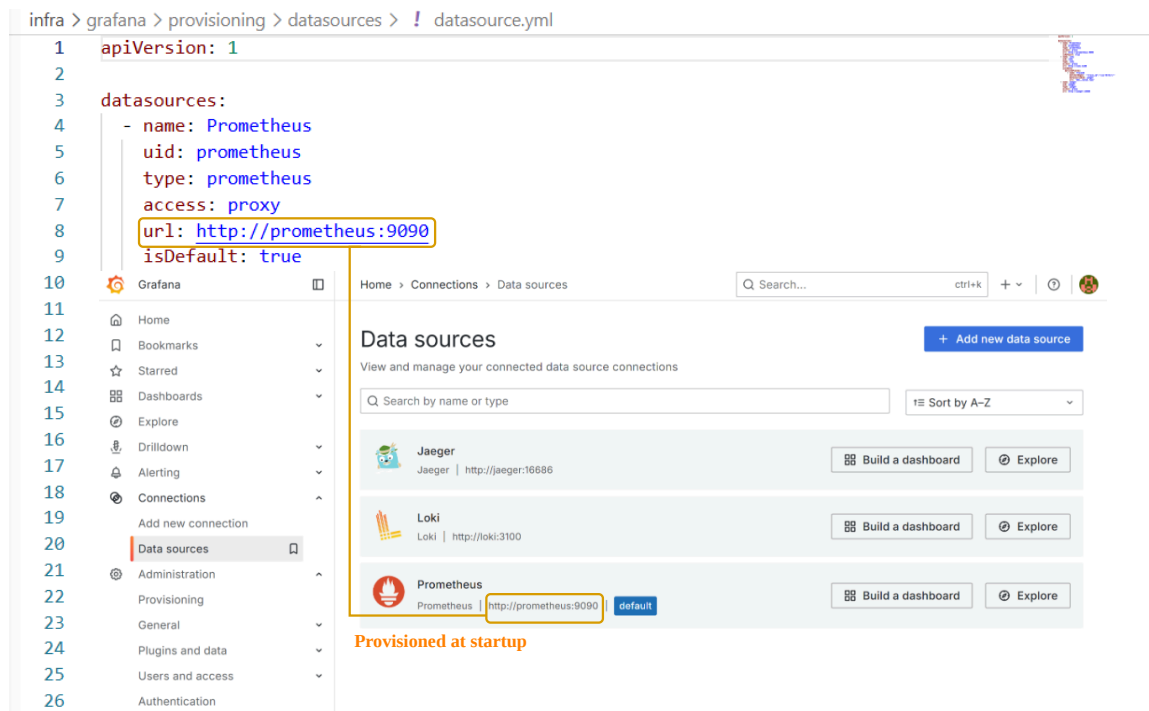
2.5 Visualizing Metrics with Grafana

Sau khi các operational signals đã được kiểm tra trên Prometheus, Grafana được sử dụng để tập hợp chúng vào cùng một dashboard và theo dõi sự thay đổi theo thời gian.

Connecting Grafana to Prometheus

Mở Grafana tại <http://localhost:3000> và đăng nhập bằng tài khoản **admin** với mật khẩu **grafana**. Sau khi đăng nhập, cấu hình Prometheus làm data source để Grafana có thể truy vấn metrics.

Khi kết nối thành công, Grafana có thể gửi PromQL query đến Prometheus và sử dụng kết quả trả về để xây dựng các panel trên dashboard.

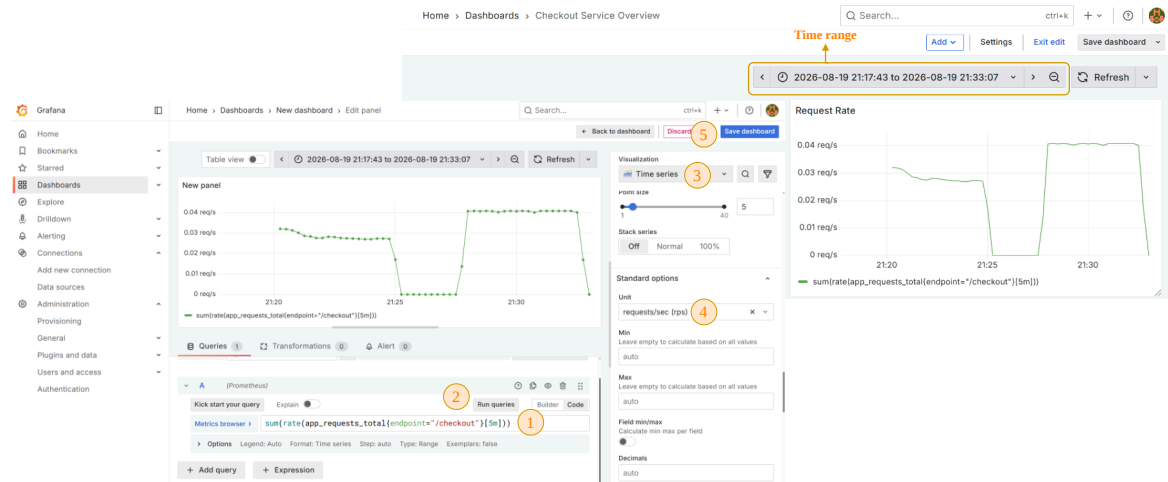


Hình 22: Prometheus được cấu hình làm data source trong Grafana.

Building the Metrics Dashboard

Một dashboard gồm nhiều panel, trong đó mỗi panel biểu diễn một signal hoặc một góc nhìn cụ thể của hệ thống. Để minh họa quy trình, ta chọn Request Rate sử dụng làm panel mẫu.

Trong Grafana, tạo một dashboard mới, chọn *Add panel* và chọn Prometheus làm data source. Sau đó sử dụng lại PromQL query Request Rate đã xây dựng ở Mục 2.4, chọn Time series làm visualization, đặt tên panel là *Request Rate*, cấu hình đơn vị requests per second và lưu panel.

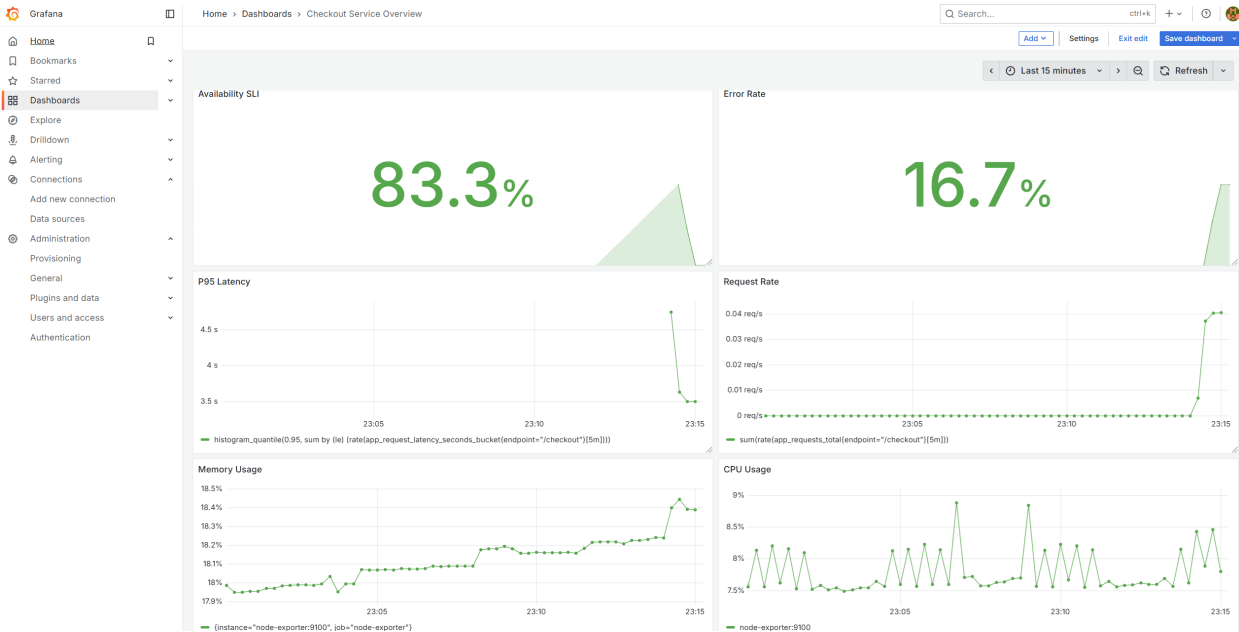


Hình 23: Xây dựng panel Request Rate bằng PromQL và Time series visualization.

Time range ở phía trên dashboard xác định khoảng thời gian được hiển thị. Khi thay đổi time range, Grafana truy vấn lại Prometheus cho khoảng thời gian tương ứng, giúp quan sát signal trong vài phút gần nhất hoặc mở rộng sang một khoảng dài hơn khi cần phân tích xu hướng.

Sau khi hoàn thành panel Request Rate, các query đã xây dựng ở Mục 2.4 được tái sử dụng để bổ sung các panel còn lại. Mỗi panel sử dụng PromQL tương ứng với tín hiệu cần quan sát và lựa chọn visualization, unit phù hợp.

Panel	PromQL	Visualization	Unit
Request Rate	<code>sum(rate(app_requests_total{endpoint="/checkout"}[5m]))</code>	Time series	req/s
Error Rate	<code>sum(rate(app_requests_total{endpoint="/checkout",status=~"5.."}[5m]))/sum(rate(app_requests_total{endpoint="/checkout"}[5m]))</code>	Stat	Percent (0 - 1)
P95 Latency	<code>histogram_quantile(0.95,sumby(le)(rate(app_request_latency_seconds_bucket{endpoint="/checkout"}[5m])))</code>	Time series	seconds
CPU Usage	<code>100*(1-avgby(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])))</code>	Time series	Percent (0 - 100)
Memory Usage	<code>100*(1-node_memory_MemAvailable_bytes/node_memory_MemTotal_bytes)</code>	Time series	Percent (0 - 100)
Availability SLI	<code>sum(rate(app_requests_total{endpoint="/checkout",status=~"2.."}[5m]))/sum(rate(app_requests_total{endpoint="/checkout"}[5m]))</code>	Stat	Percent (0 - 1)



Hình 24: Dashboard tập hợp các operational signals của checkout service trên cùng một giao diện.

Dashboard hoàn chỉnh cho phép quan sát nhiều khía cạnh của service trong cùng một time range. Request Rate phản ánh Traffic, Error Rate phản ánh Errors, P95 Latency thể hiện Latency, còn CPU Usage và Memory Usage cho thấy mức sử dụng tài nguyên, đồng thời hỗ trợ đánh giá saturation khi kết hợp với các tín hiệu chờ hoặc queue. Availability SLI cho biết tỷ lệ request thành công và hỗ trợ đánh giá reliability của service.

Sau khi dashboard được hoàn thiện, có thể tạo thêm traffic và một số request lỗi cho checkout để quan sát cách các panel thay đổi theo thời gian. Việc đặt các tín hiệu trên cùng một dashboard giúp đối chiếu Request Rate, Error Rate, P95 Latency và resource usage trong cùng một khoảng thời gian, từ đó thấy rõ cách raw metrics được Prometheus thu thập, PromQL chuyển thành operational signals và Grafana trực quan hóa các tín hiệu đó.

3 Logs with Loki and Alloy

3.1 From Metrics to Logs

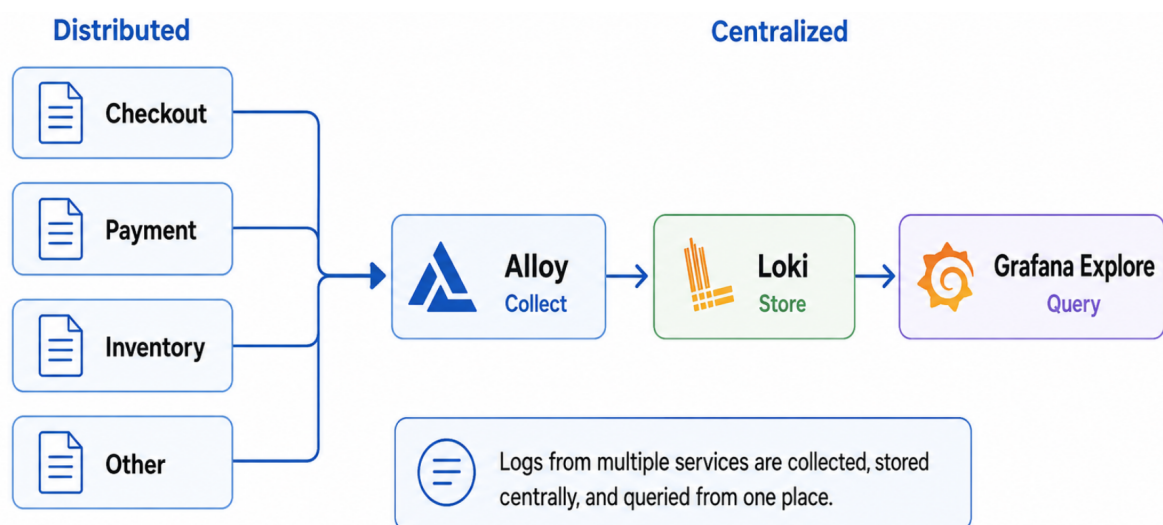
Ở Chương 2, Prometheus và Grafana đã giúp engineer theo dõi các operational signals như Request Rate, Error Rate, P95 Latency, CPU Usage, Memory Usage và Availability SLI. Những tín hiệu này giúp phát hiện khi trạng thái của hệ thống bắt đầu thay đổi, chẳng hạn Error Rate tăng hoặc P95 Latency cao hơn bình thường.

Tuy nhiên, Metrics chủ yếu phản ánh trạng thái tổng hợp của hệ thống theo thời gian. Khi dashboard cho thấy checkout service đang có tỷ lệ lỗi tăng, thông tin đó chưa cho biết sự kiện cụ thể nào đã xảy ra trong quá trình application xử lý request.

Để trả lời câu hỏi *What happened?*, engineer cần kiểm tra logs. Một log entry có thể ghi lại những sự kiện cụ thể như database timeout, request không hợp lệ hoặc external service trả về lỗi.

Trong một application gồm nhiều service và container, logs có thể được tạo ra ở nhiều thành phần khác nhau. Nếu phải mở từng container hoặc kiểm tra từng nguồn log riêng lẻ, việc tìm đúng sự kiện liên quan sẽ trở nên khó khăn hơn khi số lượng thành phần tăng lên.

Vì vậy, logs thường được đưa về một hệ thống centralized logging, nơi dữ liệu từ nhiều nguồn có thể được tìm kiếm và truy vấn tập trung.



Hình 25: Logs từ nhiều application service và container được Alloy thu thập, chuyển đến Loki để lưu trữ và sau đó được truy vấn tập trung bằng Grafana.

Trong tutorial này, centralized logging sử dụng Grafana Alloy, Loki và Grafana như minh họa trong Hình 25. Các phần tiếp theo sẽ đi sâu vào cách log data được tổ chức, thu thập và truy vấn để tìm đúng ERROR log của service trong khoảng thời gian xảy ra sự cố.

3.2 Understanding Log Data

Trước khi logs được thu thập và lưu trữ tập trung, cần hiểu log được biểu diễn dưới dạng nào và những thông tin nào giúp quá trình tìm kiếm, phân tích và điều tra sự cố hiệu quả hơn. Hai cách biểu diễn phổ biến là **unstructured logging** và **structured logging**.

Unstructured and Structured Log Representation

Unstructured log: event represented as a text line

Structured log: event represented as structured fields

Hình 26: Cùng một sự kiện có thể được ghi dưới dạng một chuỗi text hoặc được tổ chức thành các field có cấu trúc.

Unstructured and Structured Logs

Một **unstructured log** ghi thông tin của sự kiện dưới dạng một chuỗi text tự do. Cách này đơn giản và phù hợp với những trường hợp chưa cần xử lý log theo nhiều thuộc tính riêng biệt.

Ví dụ: `2026-08-17 10:05:23 ERROR checkout Database connection timeout`

Dòng log trên vẫn chứa các thông tin như thời gian, mức độ lỗi, service và message, nhưng tất cả nằm trong cùng một chuỗi text. Khi cần lấy riêng một thuộc tính, hệ thống thường phải dựa vào text matching, pattern parsing hoặc regular expression. Nếu format của log thay đổi, các pattern dùng để xử lý dữ liệu cũng có thể phải thay đổi theo.

Ngược lại, **structured log** tổ chức cùng thông tin thành các field có cấu trúc rõ ràng, thường sử dụng format như JSON hoặc key-value.

```
{
  "timestamp": "2026-08-17T10:05:23Z",
  "level": "ERROR",
  "service_name": "checkout",
  "trace_id": "7f31a9",
  "message": "Database connection timeout"
}
```

Khi mỗi thuộc tính được biểu diễn thành một field riêng, hệ thống có thể parse, filter và query từng field trực tiếp thay vì phải suy luận cấu trúc từ một chuỗi text.

Cần phân biệt **metadata đi kèm log** với **cấu trúc của nội dung log**. Một log dạng text vẫn có thể được gắn thêm thông tin như `service="payment"` hoặc `environment="prod"` để cho biết log đến từ đâu. Tuy nhiên, nếu nội dung vẫn là `Payment failed for user 123, order ORD-456`, thì đây vẫn là unstructured log vì các thông tin như user và order vẫn nằm trong một chuỗi text. Với structured logging, những thông tin này có thể được tách thành các field như `user_id`, `order_id` hoặc `reason` để truy vấn trực tiếp.

Log Levels and Metadata

Dù log được ghi dưới dạng unstructured hay structured, mỗi log entry vẫn cần đủ context để engineer biết sự kiện xảy ra khi nào, ở đâu và trong trạng thái nào. Context này thường được thể hiện thông qua các metadata đi kèm log entry.

Một số field thường gặp gồm:

Field	Vai trò
<code>timestamp</code>	Xác định thời điểm sự kiện xảy ra
<code>level</code>	Biểu thị mức độ nghiêm trọng của sự kiện
<code>service_name</code>	Xác định service tạo ra log
<code>trace_id</code>	Liên kết log với request hoặc trace tương ứng
<code>message</code>	Mô tả sự kiện đã xảy ra

Bảng 2: Các field thường gặp trong một log entry.

Trong structured logging, các thông tin này thường được biểu diễn thành những field riêng biệt. Với unstructured logging, chúng vẫn có thể xuất hiện trong log line nhưng thường nằm chung trong một chuỗi text.

Log level là một loại metadata dùng để phân loại mức độ quan trọng của sự kiện. Các mức phổ biến gồm:

- **DEBUG**: cung cấp thông tin chi tiết về quá trình thực thi, thường được sử dụng khi troubleshooting hoặc debugging.
- **INFO**: ghi nhận các hoạt động bình thường của hệ thống.
- **WARN**: biểu thị tình huống bất thường hoặc cần chú ý nhưng chưa nhất thiết làm quá trình xử lý thất bại.
- **ERROR**: cho biết một operation hoặc quá trình xử lý đã gặp lỗi và có thể cần được điều tra.

Ví dụ, khi các metadata được biểu diễn dưới dạng structured fields, một log entry có thể có dạng:

```
{  
  "timestamp": "2026-08-17T10:05:23Z",
```



```
"level": "ERROR",
"service_name": "checkout",
"trace_id": "7f31a9",
"message": "Database connection timeout"
}
```

Từ các field này, ta có thể thu hẹp dữ liệu theo `service_name`, khoảng thời gian và `level`, sau đó sử dụng `trace_id` nếu cần tiếp tục điều tra request liên quan.

3.3 Collecting Logs with Grafana Alloy

Trong logging architecture ở phần trước, Grafana Alloy đóng vai trò collector giữa log source và Loki. Alloy nhận log từ các source được cấu hình, có thể xử lý hoặc bổ sung metadata, sau đó forward các log entries đến Loki.

Một Alloy configuration thường xác định ba phần chính:

- **Source** xác định nơi log được lấy vào.
- **Processing** thực hiện các bước xử lý hoặc bổ sung metadata khi cần.
- **Output** xác định nơi log được gửi đến.

Alloy Configuration

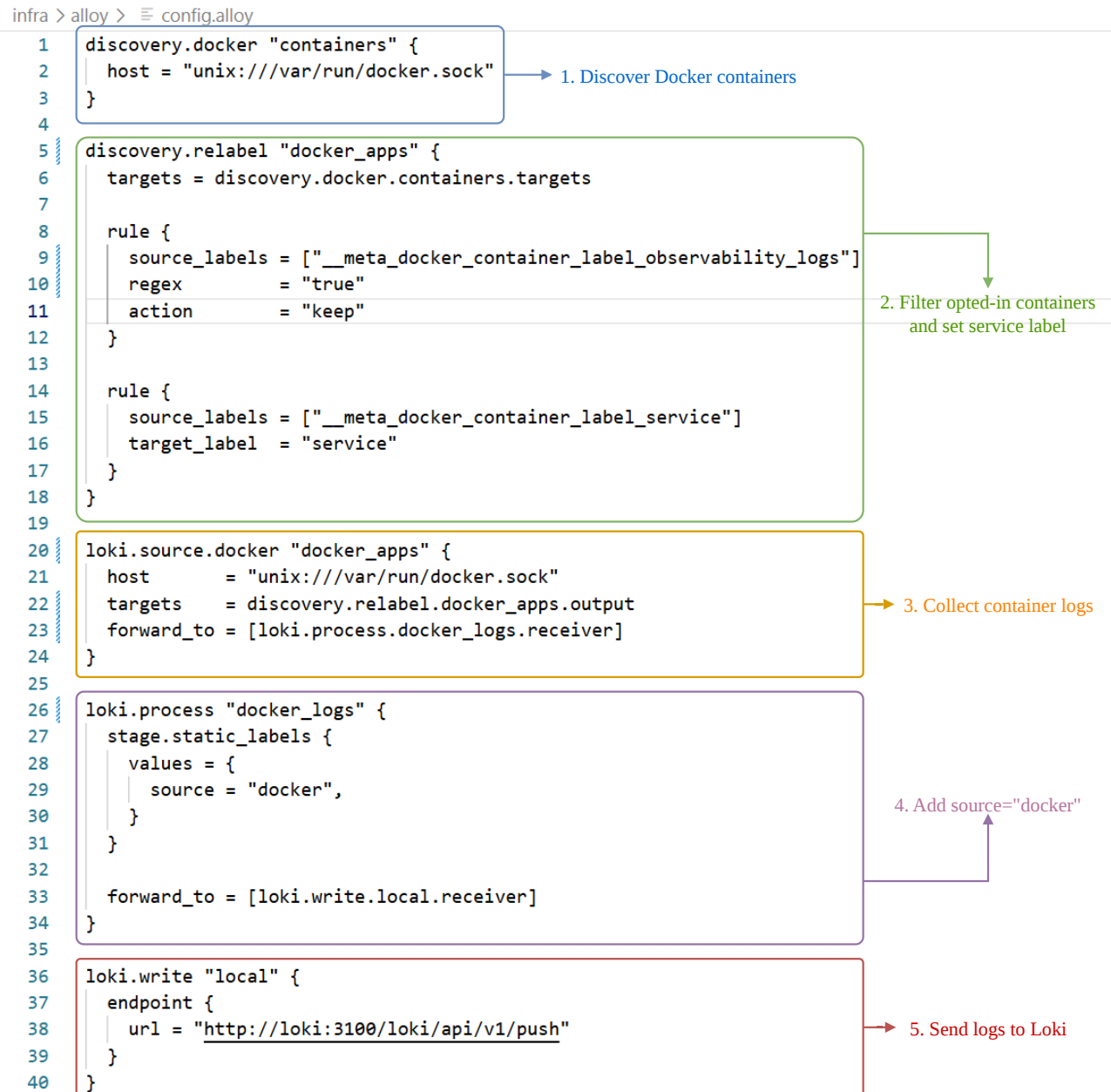
Trong repository, Alloy configuration đã được chuẩn bị sẵn để thu thập container logs và gửi chúng đến Loki. Mỗi component được khai báo theo dạng `<component type> "instance name"`, sau đó được liên kết với các component khác thông qua các input và output tương ứng.

Hai field quan trọng cần chú ý là:

- **targets**: xác định danh sách target mà component sử dụng làm input.
- **forward_to**: xác định receiver của component tiếp theo sẽ nhận log entries.

Hình 27 minh họa toàn bộ configuration hiện tại. `discovery.docker` tìm các Docker containers đang chạy. Kết quả discovery được chuyển sang `discovery.relabel`, nơi chỉ các containers có label `observability.logs="true"` được giữ lại và Docker label `service` được chuyển thành label `service` để phục vụ truy vấn logs. Việc lựa chọn labels cần cân nhắc đến cardinality, tương tự khái niệm đã được giới thiệu với Prometheus.

Các targets sau khi được lọc được chuyển đến `loki.source.docker` để thu thập container logs. Log entries sau đó đi qua `loki.process`, nơi bổ sung label tĩnh `source="docker"`, trước khi `loki.write` gửi dữ liệu đến Loki.



Hình 27: Các component và data flow trong Alloy configuration từ Docker discovery đến Loki.

Từ configuration trong hình, có thể lần theo trực tiếp cách các component được nối với nhau. Chẳng hạn, `loki.source.docker "docker_apps"` nhận `targets` từ `discovery.relabel.docker_apps.output` thông qua field `targets`. Sau khi thu thập logs, component này sử dụng `forward_to` để chuyển log entries đến `loki.process.docker_logs.receiver`.

Tương tự, sau bước processing, `loki.process "docker_logs"` tiếp tục forward các log entries đến `loki.write.local.receiver`. Nhờ đó, khi đọc một Alloy configuration, engineer có thể lần theo các tham chiếu giữa `targets`, `output` và `receiver` để xác định log được lấy từ đâu, đi qua những component nào và cuối cùng được gửi đến đâu.

3.4 Storing Logs with Loki

Sau khi Alloy thu thập và forward log entries, dữ liệu được gửi đến Loki để tổ chức, lưu trữ và phục vụ truy vấn. Nếu Alloy đảm nhiệm việc đưa log vào centralized logging pipeline, Loki cung cấp lớp lưu trữ và truy vấn cho dữ liệu log.

Loki Overview

Loki là hệ thống log aggregation nhận log từ các collector như Alloy và cung cấp dữ liệu cho các công cụ truy vấn hoặc visualization. Grafana có thể sử dụng Loki như một data source để tìm kiếm log bằng LogQL và hiển thị kết quả trong Explore hoặc dashboard.

Về cách tổ chức dữ liệu, Loki có một số điểm tương đồng với Prometheus:

Prometheus	Loki
Metrics	Logs
Time series	Log streams
PromQL	LogQL
Labels xác định series	Labels xác định streams
Numeric samples	Log entries

Bảng 3: So sánh cách tổ chức dữ liệu giữa Prometheus và Loki.

Trong Prometheus, một label set xác định một time series. Loki áp dụng ý tưởng tương tự cho logs, trong đó một tập labels xác định một log stream và stream đó chứa các log entries xuất hiện theo thời gian.

Một điểm quan trọng là Loki sử dụng labels để xác định phạm vi log cần truy vấn thay vì index toàn bộ nội dung của từng log entry. Sau khi các stream phù hợp được xác định, nội dung bên trong stream mới được xử lý khi query.

Labels and Log Streams

Trong Loki, labels là các cặp key-value dùng để mô tả nguồn hoặc context của log. Những log entries có cùng một label set được nhóm vào cùng một log stream.

Ví dụ: `{service_name="checkout", environment="prod"}`

Label set trên có thể xác định stream chứa logs của service checkout trong environment prod.

Loki Labels and Log Streams

Label set → Log stream → Log entries over time

Hình 28: Loki sử dụng labels để nhóm các log entries có cùng context vào một log stream.

Mỗi entry trong stream vẫn giữ timestamp và nội dung log riêng. Labels được dùng để thu hẹp phạm vi dữ liệu cần tìm, còn thông tin chi tiết của từng sự kiện vẫn nằm trong log entry.

Ví dụ, cùng một stream có thể chứa:

```
10:05:21 INFO Checkout request received
10:05:23 ERROR Database connection timeout
10:05:25 INFO Request completed
```

Ba log entries trên cùng thuộc một stream vì chúng chia sẻ cùng label set, nhưng mỗi entry biểu diễn một sự kiện khác nhau tại một thời điểm khác nhau.

Choosing Loki Labels

Không phải mọi field xuất hiện trong log đều nên trở thành label. Khi một giá trị được dùng làm label, mỗi combination khác nhau của label values có thể tạo ra một log stream mới. Vì vậy, việc lựa chọn labels cần cân nhắc đến cardinality, khái niệm đã được giới thiệu với Prometheus.

Các field có số lượng giá trị tương đối nhỏ và ổn định như **service_name**, **environment** hoặc **container** thường phù hợp hơn để làm labels. Ngược lại, những field thay đổi gần như theo từng request như **trace_id**, **request_id**, **user_id** hoặc **timestamp** thường có high cardinality và không nên được dùng làm labels. Ví dụ, nếu **trace_id** được dùng làm label:

```
{service_name="checkout", trace_id="a123"}
{service_name="checkout", trace_id="b456"}
{service_name="checkout", trace_id="c789"}
```

Mỗi **trace_id** tạo ra một label set khác nhau. Khi số lượng request tăng, số lượng log streams cũng có thể tăng nhanh.

Điều đó không có nghĩa **trace_id** không nên xuất hiện trong log. Field này vẫn có thể được giữ

trong structured log để phục vụ quá trình điều tra:

```
{
  "level": "ERROR",
  "service_name": "checkout",
  "trace_id": "7f31a9",
  "message": "Database connection timeout"
}
```

Ở đây, `service_name` có thể được dùng để thu hẹp log về service checkout, trong khi `trace_id` chỉ được parse và filter khi cần tìm một request cụ thể.

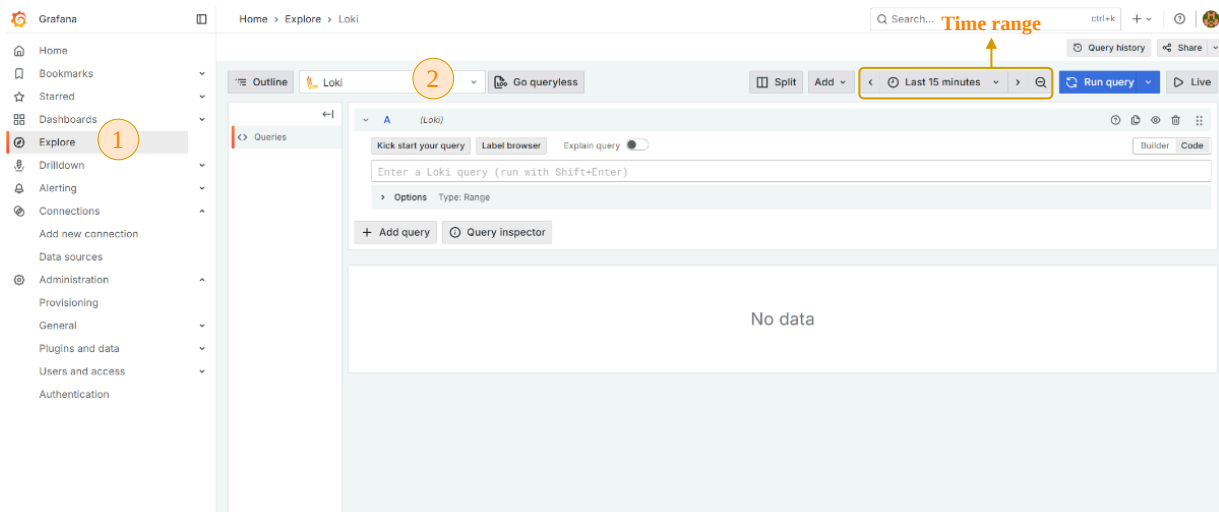
3.5 Hands-on: Investigating Checkout Errors with LogQL

Phần này thực hành sử dụng Grafana Explore và LogQL để điều tra lỗi của service checkout. Metrics dashboard đã xác định khoảng thời gian có dấu hiệu bất thường, vì vậy bước tiếp theo là tìm các ERROR logs xuất hiện trong khoảng thời gian đó và kiểm tra thông tin liên quan.

Mục tiêu của bài thực hành là xác định relevant log entry, xem context xung quanh lỗi và ghi nhận `trace_id` để tiếp tục điều tra ở Chương 4.

Open Logs in Grafana Explore

Mở Grafana Explore và chọn Loki làm data source. Sau đó đặt time range trùng với incident window đã xác định từ metrics.



Hình 29: Grafana Explore được cấu hình để truy vấn Loki trong khoảng thời gian xảy ra sự cố.

Explore là nơi chạy LogQL và inspect các log entries trả về. Time range cần được đặt đúng để tránh truy vấn một lượng log lớn không liên quan.

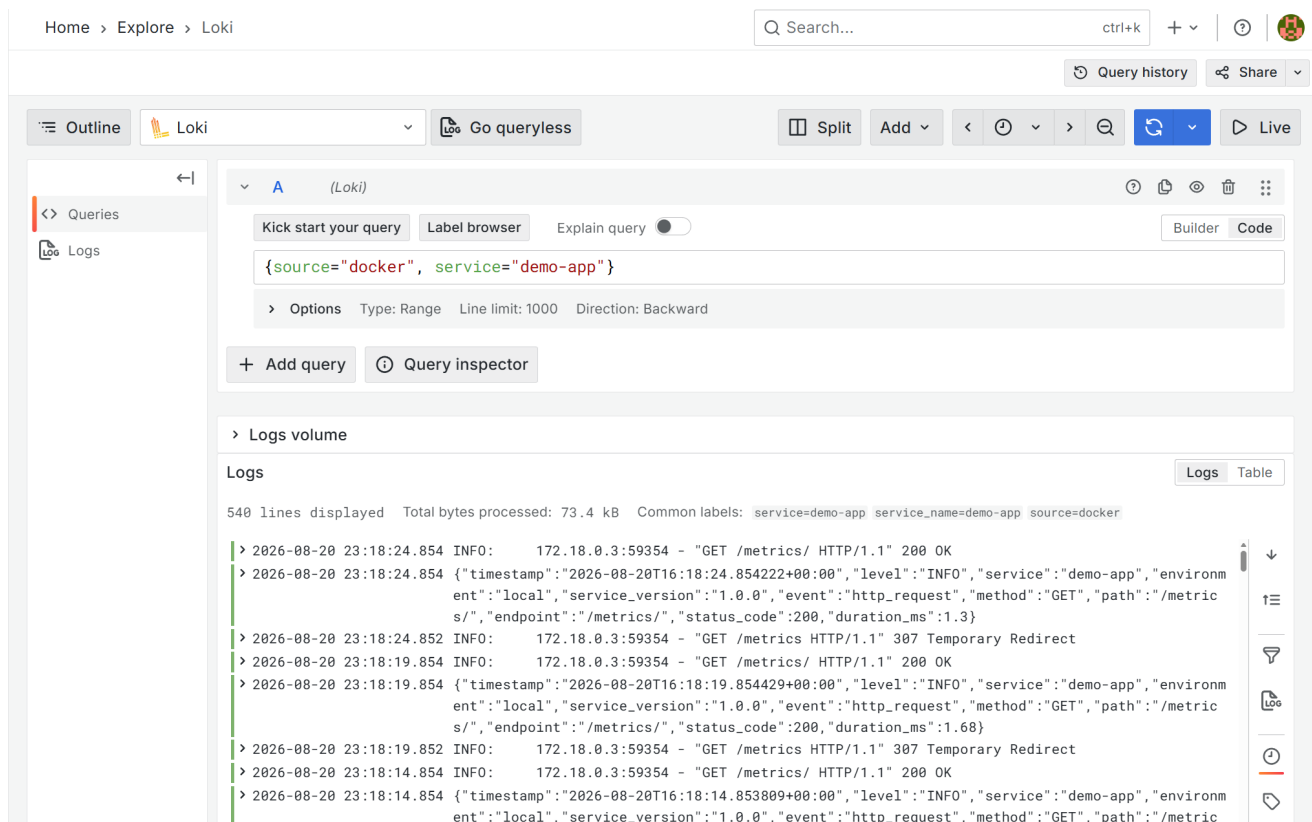
Select the Checkout Log Stream

Bắt đầu bằng cách chọn log stream của checkout service:

```
{source="docker", service="demo-app"}
```

Phần `{...}` là stream selector, sử dụng labels để xác định các log streams cần truy vấn. Query trên chọn logs có `source="docker"` và `service="demo-app"`.

Trong lab này, `demo-app` xử lý các request tới endpoint `/checkout`, vì vậy log stream này là điểm bắt đầu để điều tra các lỗi phát sinh trong quá trình checkout.



Hình 30: Log entries của `demo-app` được trả về sau khi chọn stream bằng các labels `source` và `service`.

Sau khi xác nhận đúng log stream, bước tiếp theo là thu hẹp các log entries liên quan đến lỗi cần điều tra.

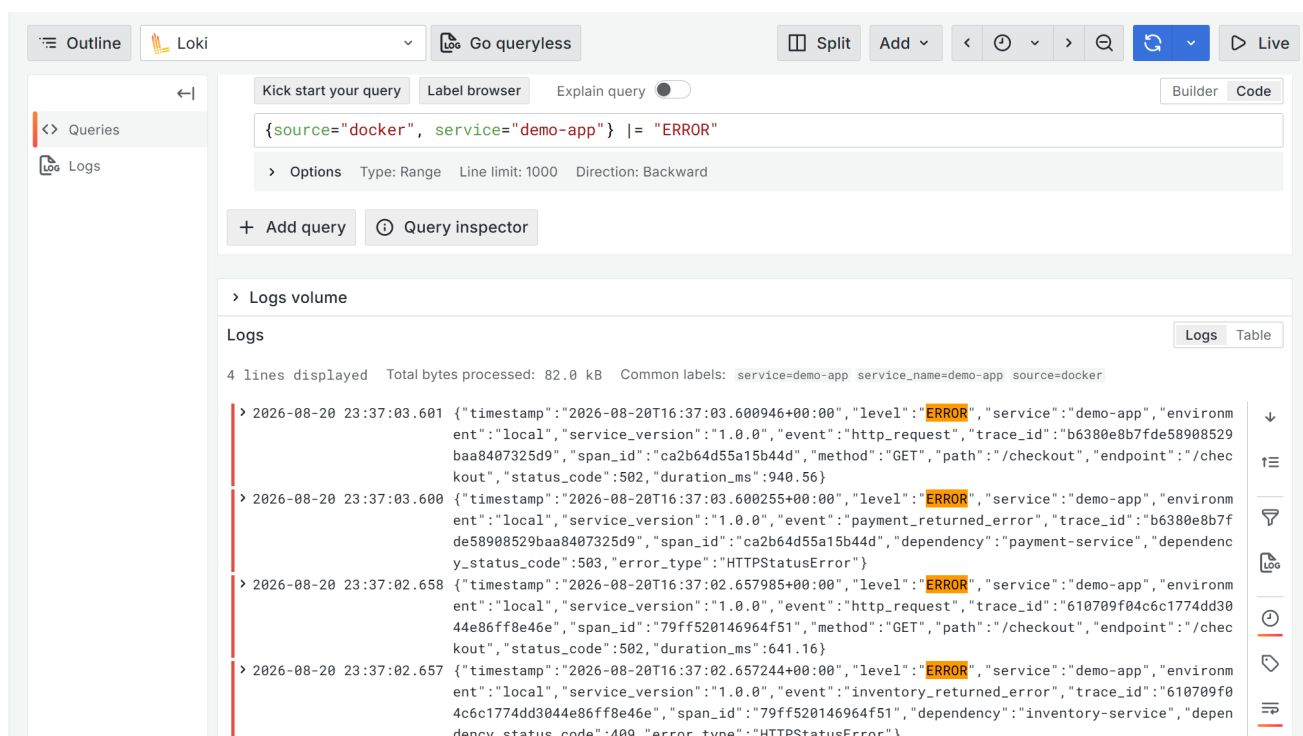
Filter Relevant Log Lines

Khi log volume lớn, có thể tiếp tục thu hẹp kết quả bằng line filter:

```
{source="docker", service="demo-app"} |= "ERROR"
```

Operator `!=` giữ lại các log lines chứa chuỗi được chỉ định. Query trên vì vậy chỉ trả về logs của `demo-app` có chứa chuỗi `ERROR`.

Cách lọc này phù hợp khi cần tìm nhanh các log entries liên quan đến lỗi, đặc biệt với unstructured logs hoặc trước khi parse structured log content.



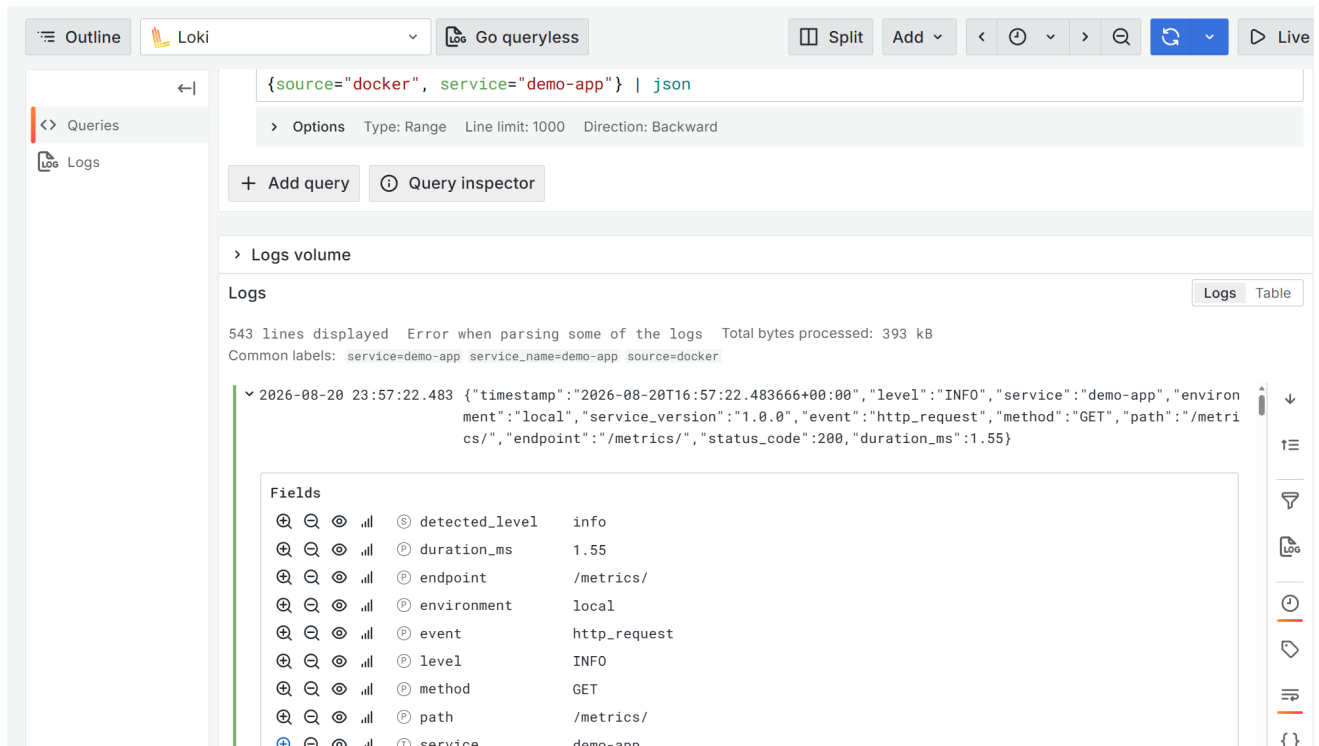
Hình 31: Kết quả sau khi sử dụng line filter để giữ các log lines chứa `ERROR`.

Parse Structured Logs

Ở bước trước, line filter chỉ tìm chuỗi xuất hiện trong nội dung log. Với structured logs, LogQL có thể parse nội dung thành các field để truy vấn trực tiếp:

```
{source="docker", service="demo-app"} | json
```

LogQL hỗ trợ nhiều parser như `json`, `logfmt`, `pattern` và `regexp`, tùy theo format của log. Trong lab này, application ghi structured logs dưới dạng JSON, vì vậy `| json` được sử dụng để parse log body và trích xuất các field như `level`, `event`, `trace_id`, `path` và `status_code`.



Hình 32: Các field được trích xuất từ structured log bằng JSON parser trong Grafana Explore.

Trong thực tế, cùng một log stream có thể chứa log entries ở nhiều format khác nhau. Khi sử dụng `| json`, chỉ những entries có JSON hợp lệ mới được parse thành các field tương ứng, trong khi các entries khác vẫn được giữ nguyên để tiếp tục quan sát hoặc xử lý theo cách phù hợp.

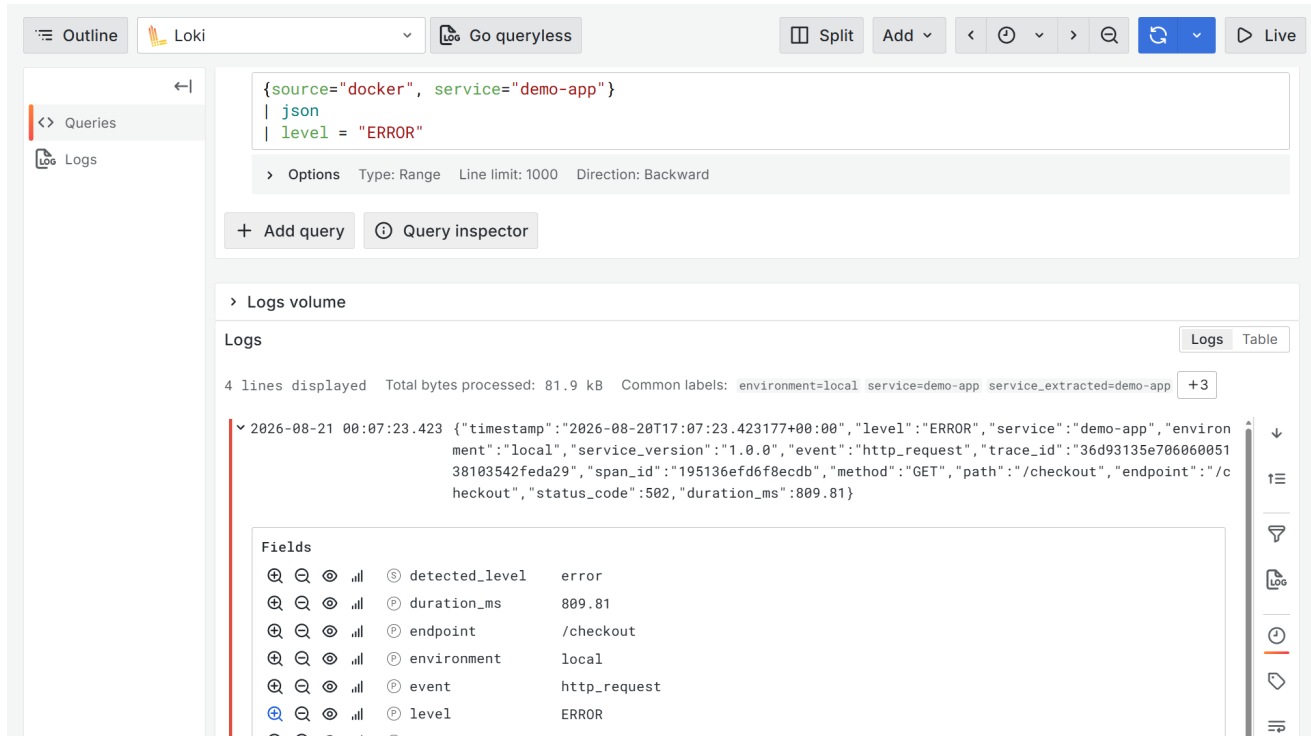
Sau khi parse, việc tìm lỗi có thể dựa trực tiếp vào giá trị của từng field thay vì tìm chuỗi **ERROR** trong toàn bộ log line.

Filter **ERROR** Logs by Field

Sau khi structured log được parse, có thể filter trực tiếp theo giá trị của field `level`:

```
{source="docker", service="demo-app"}
| json
| level = "ERROR"
```

Khác với line filter ở bước trước, điều kiện này chỉ giữ lại các log entries có field `level` mang giá trị **ERROR**. Time range trên Grafana Explore được giữ trong incident window đã xác định từ metrics để giới hạn kết quả vào khoảng thời gian cần điều tra.

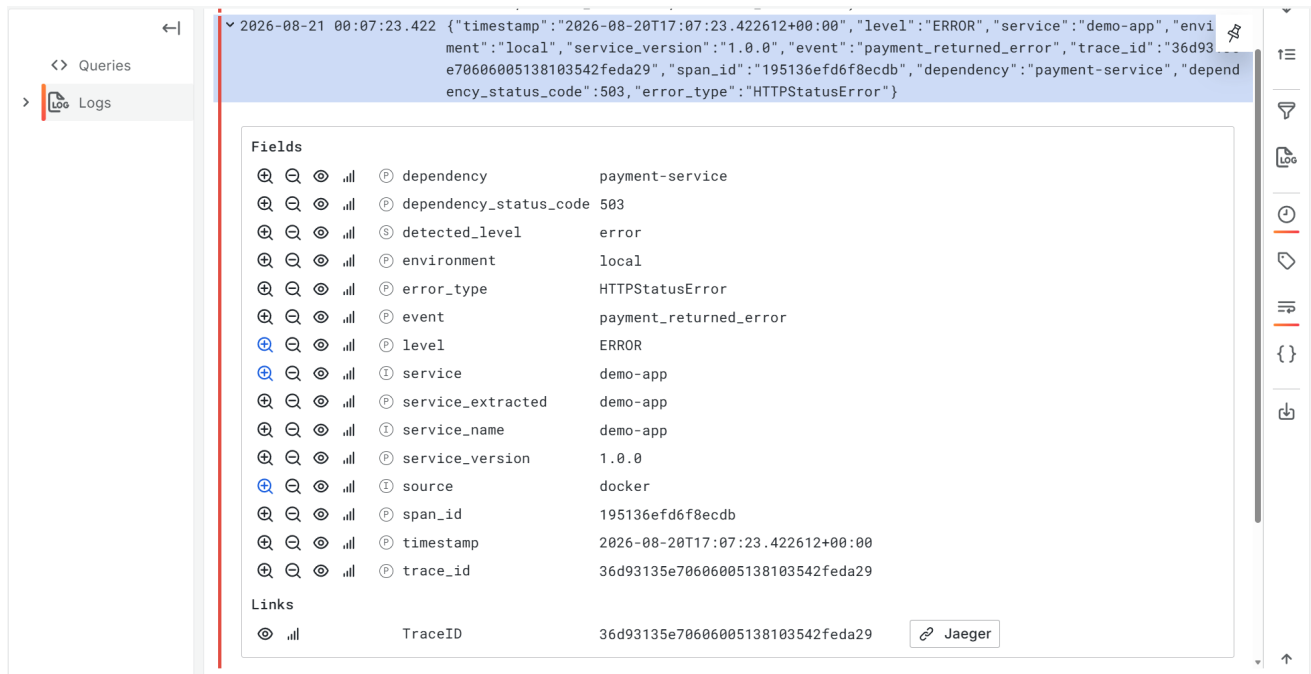


Hình 33: Các log entries có `level="ERROR"` trong incident window.

Từ các ERROR entries, engineer có thể kiểm tra thêm những field như `event`, `status_code`, `dependency` và `trace_id` để thu hẹp nguyên nhân và xác định request cần tiếp tục điều tra.

Inspect the Error Context

Chọn một ERROR entry và mở rộng log để kiểm tra các field liên quan đến sự kiện. Với event `payment_returned_error`, các field như `dependency`, `dependency_status_code`, `error_type`, `span_id` và `trace_id` cung cấp thêm context về dependency và request đang gặp lỗi.



Hình 34: Các field của event `payment_returned_error` được mở rộng trong Grafana Explore.

Ngoài việc mở rộng một log entry để xem các field đã parse, Grafana Explore còn hỗ trợ *Show context* để xem các log entries xuất hiện trước và sau sự kiện trong cùng log stream khi cần thêm ngữ cảnh.

Trong các field trên, `trace_id` cho phép liên kết log entry với trace của cùng request. Giá trị này sẽ được sử dụng ở Chương 4 để tiếp tục xác định request đã đi qua những operation nào và lỗi xuất hiện tại đâu trên request path.

Create a Numeric Signal from Logs

LogQL không chỉ dùng để tìm và đọc log entries. Sau khi structured logs được parse thành các field, những field này cũng có thể được sử dụng để aggregate log data thành numeric signal.

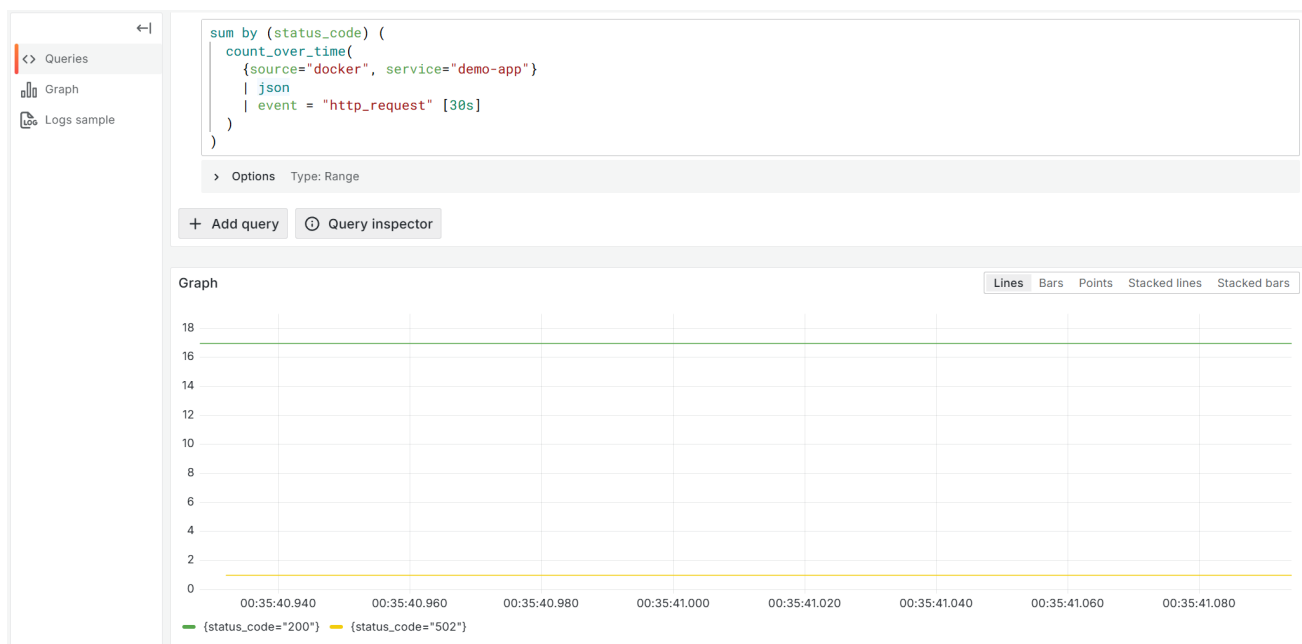
Ví dụ, query sau đếm số HTTP request logs theo từng `status_code` trong cửa sổ 30 giây:

```
sum by (status_code) (
  count_over_time(
    {source="docker", service="demo-app"}
    | json
    | event = "http_request" [30s]
  )
)
```

`count_over_time` đếm số log entries phù hợp xuất hiện trong 30 giây gần nhất. `sum by (status_code)` sau đó nhóm kết quả theo HTTP status code, tạo một time series riêng cho mỗi giá trị như 200 hoặc 502.

Cách xử lý này tương tự range window và aggregation đã sử dụng với PromQL ở Chương 2: dữ

liệu trong một khoảng thời gian được tổng hợp thành numeric signal để theo dõi theo thời gian. Điểm khác là nguồn dữ liệu ban đầu ở đây là log entries thay vì metric samples.



Hình 35: Số HTTP request log entries được nhóm theo `status_code` trong cửa sổ 30 giây.

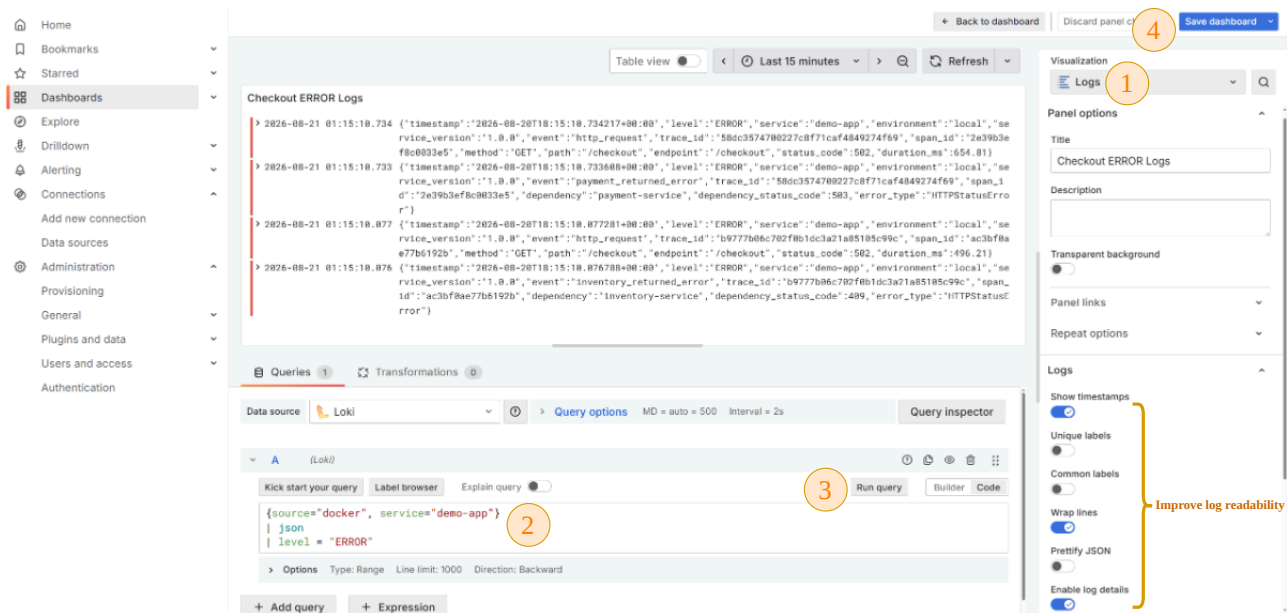
Trong kết quả trên, phần lớn request kết thúc với `status_code="200"`, trong khi một số request trả về 502. Qua đó, các log entries rời rạc được chuyển thành numeric signal có thể sử dụng cho visualization, dashboard hoặc tiếp tục phân tích theo thời gian.

Add the Query to a Logs Panel

Sau khi query đã được kiểm tra trong Explore, có thể reuse query đó trong Grafana dashboard để theo dõi các ERROR logs thường xuyên hơn.

Tạo một panel sử dụng Loki làm data source, chọn *Logs* visualization và sử dụng lại query:

```
{source="docker", service="demo-app"}
| json
| level = "ERROR"
```



Hình 36: ERROR log entries được hiển thị trong Grafana Logs panel.

Sau khi panel được tạo, cùng query đã kiểm tra trong Explore có thể được đưa vào dashboard và hiển thị bằng *Logs* visualization. Panel sử dụng Loki làm data source, query lọc các log entries có `level="ERROR"` và kết quả được hiển thị trực tiếp để theo dõi thường xuyên hơn.

Cách reuse này cho phép Explore phục vụ quá trình điều tra ad-hoc, trong khi dashboard cung cấp một góc nhìn phù hợp cho việc theo dõi liên tục.

4 Traces with OpenTelemetry and Jaeger

4.1 From Logs to Distributed Tracing

Trong distributed system, nơi một lỗi được quan sát thấy chưa chắc là nơi lỗi bắt đầu. Một request `/checkout` có thể trả về HTTP 502 tại `demo-app`, nhưng nguyên nhân thực tế có thể nằm ở downstream service hoặc external dependency mà request đã gọi trong quá trình xử lý. Distributed Tracing cung cấp góc nhìn end-to-end để theo dõi cùng một request xuyên qua các thành phần này và thu hẹp operation liên quan đến latency hoặc failure.

From Logs to Traces

Ở Chương 2, Metrics giúp phát hiện khi trạng thái của checkout service thay đổi, chẳng hạn Error Rate tăng hoặc P95 Latency cao hơn bình thường. Sang Chương 3, Logs cung cấp thêm context về những sự kiện xảy ra trong cùng incident window. Ví dụ, một ERROR log có thể cho thấy `demo-app` nhận lỗi từ `payment-service`:

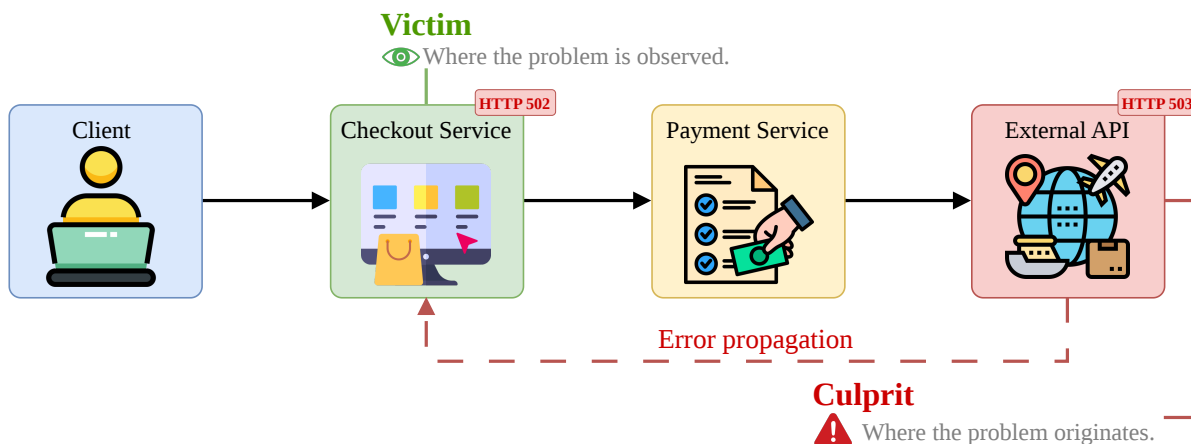
```
{
  "timestamp": "2026-08-17T10:07:23Z",
  "level": "ERROR",
  "service_name": "demo-app",
  "event": "payment_returned_error",
  "dependency": "payment-service",
  "dependency_status_code": 503,
  "trace_id": "36d93135e7060605138103542feda29"
}
```

Log entry này cho biết lỗi được ghi nhận tại `demo-app` trong lúc gọi `payment-service`. Tuy nhiên, thông tin đó chưa đủ để kết luận `payment-service` là nơi failure bắt đầu, vì service này có thể tiếp tục gọi một dependency khác ở phía sau.

Giả sử external payment API trả HTTP 503. `payment-service` nhận lỗi này và trả failure ngược về `demo-app`, sau đó `/checkout` kết thúc với HTTP 502 cho client. Trong troubleshooting, tình huống như vậy đôi khi được mô tả bằng hai vai trò:

- **Victim:** component chịu ảnh hưởng và là nơi symptom được quan sát thấy.
- **Culprit:** component khởi phát hoặc đóng góp chính vào failure.

Trong ví dụ này, `demo-app` là victim vì `/checkout` biểu hiện lỗi tại đây, còn external payment API là possible culprit nếu failure bắt đầu từ dependency đó. `payment-service` nằm giữa request path nên việc nó trả lỗi chưa đủ để kết luận đây là failure origin.



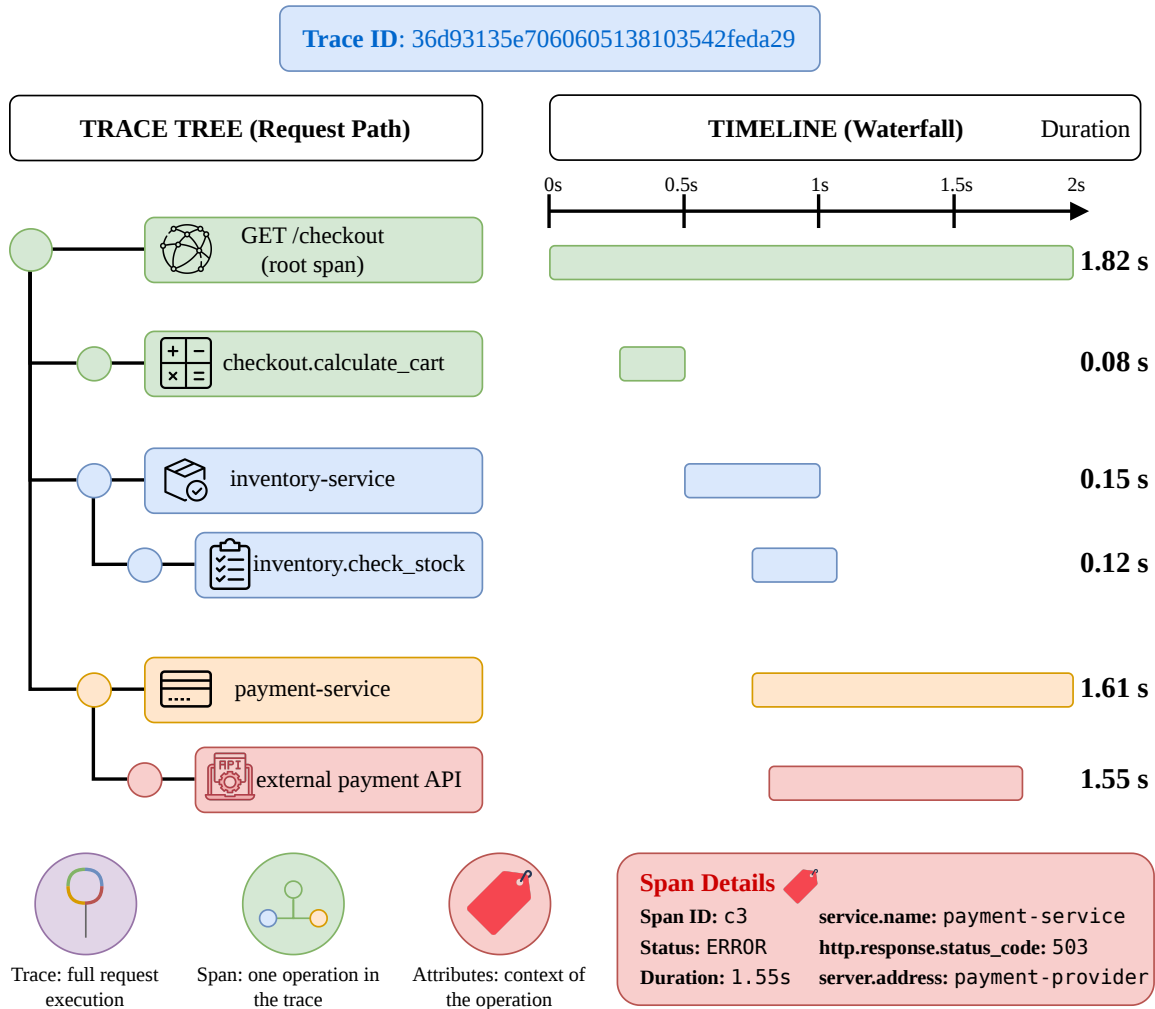
Hình 37: Một failure tại downstream dependency có thể được truyền ngược qua request path và biểu hiện thành lỗi tại upstream service.

Vấn đề còn lại là engineer cần theo dõi chính request đã thất bại đó qua các downstream components thay vì tiếp tục suy luận từ từng error riêng lẻ. `trace_id` xuất hiện trong log trở thành identifier để tìm đúng request trong tracing system và quan sát execution path của nó xuyên qua nhiều service. Execution path này được biểu diễn dưới dạng một trace, trong đó từng operation được ghi nhận bằng các span.

Trace and Span

Một Trace biểu diễn toàn bộ execution của một request từ lúc bắt đầu đến khi hoàn tất. Trong một distributed system, trace có thể trải qua nhiều service và dependency khác nhau nhưng vẫn được xem là cùng một request.

Mỗi Span biểu diễn một operation cụ thể bên trong trace. Operation đó có thể là xử lý HTTP request, gọi downstream service, thực hiện database query hoặc gọi external API. Span có thời điểm bắt đầu và kết thúc riêng, vì vậy tracing system có thể xác định duration của từng operation và quan sát thời gian được phân bổ như thế nào trên request path.



Hình 38: Distributed trace của request GET/checkout với các spans trên request path.

Trong ví dụ này, toàn bộ /checkout mất khoảng 1.82 s, trong khi nhánh payment-service chiếm 1.61 s và outbound call đến external payment API mất 1.55 s. Dựa vào đó, ta có thể thu hẹp latency từ mức toàn bộ request xuống một operation cụ thể cần được kiểm tra thêm.

Một span có duration lớn hoặc trạng thái lỗi chưa tự động đồng nghĩa với root cause. Trace trước hết giúp localize nơi latency hoặc failure tập trung trên request path, nguyên nhân phía sau vẫn cần được xác nhận bằng context và telemetry liên quan.

Trace ID, Span ID, and Span Attributes

Mỗi trace có một Trace ID dùng để nhận diện toàn bộ request, còn mỗi span có một Span ID riêng để nhận diện operation cụ thể bên trong trace. Tất cả spans thuộc cùng một trace chia sẻ Trace ID, nhưng mỗi span có Span ID khác nhau.

Bảng 4: Các field cơ bản dùng để mô tả trace và span.

Field	Ý nghĩa
<code>trace_id</code>	Nhận diện toàn bộ trace của request
<code>span_id</code>	Nhận diện một operation cụ thể trong trace
<code>duration</code>	Thời gian operation thực thi
<code>status</code>	Trạng thái của operation
<code>attributes</code>	Metadata mô tả context của operation

Nếu application ghi `trace_id` và `span_id` vào structured logs, engineer có thể liên kết một log entry với đúng request và, khi cần, với đúng operation trong trace. Nhờ đó, `trace_id` lấy được từ Logs có thể được sử dụng để tiếp tục investigation trong tracing system thay vì bắt đầu tìm kiếm lại từ đầu.

Ngoài identifier và duration, mỗi span có thể chứa attributes, tức metadata dạng key-value mô tả context của operation. Với một HTTP span, các attributes có thể gồm:

```
service.name = payment-service
http.request.method = POST
http.response.status_code = 503
server.address = payment-provider
```

Các attributes bổ sung context mà chỉ operation name hoặc duration chưa thể hiện được. Nếu outbound span của `payment-service` có duration 1.55 s, trạng thái `ERROR`, `http.response.status_code = 503` và `server.address = payment-provider`, engineer có thể đọc được rằng phần lớn thời gian của request nằm ở một call đến payment provider và call này kết thúc với lỗi. Đây là một khu vực cần điều tra thêm, nhưng chưa đủ để kết luận payment provider là root cause cuối cùng.

Kết hợp Trace ID, Span ID, duration, status và attributes giúp engineer đọc execution của một request ở mức chi tiết hơn thay vì chỉ nhìn symptom tại từng service riêng lẻ. Phần tiếp theo sẽ tập trung vào cách các spans được tạo ở nhiều service khác nhau vẫn có thể được liên kết thành cùng một request path.

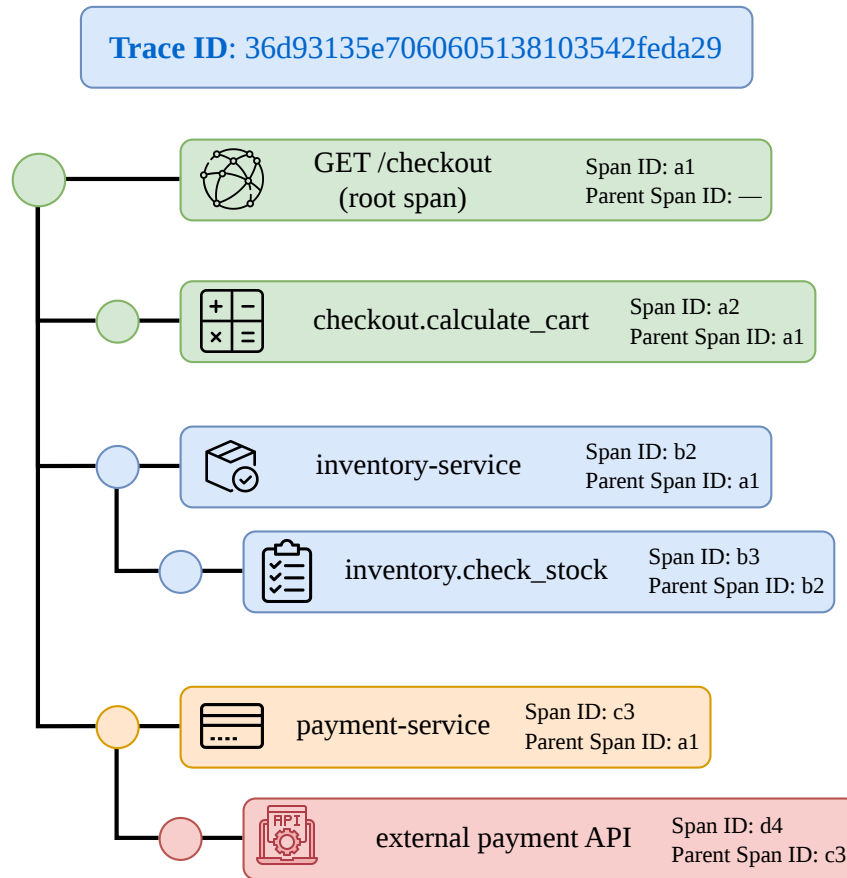
4.2 Trace Relationships and Context Propagation

Một trace không chỉ cần biết các spans nào thuộc cùng một request mà còn phải thể hiện operation nào được tạo ra từ operation nào. Khi request tiếp tục sang một downstream service, service mới cũng cần biết request đó thuộc trace nào và operation phía trước là span nào. Hai khái niệm chính để duy trì cấu trúc này là *parent-child relationship*, tức quan hệ cha-con giữa các span, và *trace context propagation*, tức quá trình truyền ngữ cảnh trace.

Parent-Child Relationships

Khi một operation tạo ra một operation khác trong cùng request, span của operation phía trước được xem là *parent span*, còn span được tạo ra từ nó là *child span*. Span bắt đầu trace và không có parent span bên trong trace đó được gọi là *root span*.

Với request `/checkout`, `GET /checkout` là root span. Các calls đến `inventory-service` và `payment-service` được tạo ra bên dưới nó, trong khi external payment API được tạo ra từ `payment-service`.



Hình 39: Quan hệ parent-child giữa các spans trong cùng một trace.

Các spans trong Hình 39 sử dụng cùng Trace ID vì thuộc cùng một request nhưng mỗi span có Span ID riêng. `payment-service` có Parent Span ID là `a1`, cho biết span này được tạo ra từ `GET /checkout`, trong khi external payment API có Parent Span ID là `c3` và được nối với span của `payment-service`.

Parent-child relationship giúp tracing system biết các spans phải được nối với nhau như thế nào. Tuy nhiên, khi request từ `GET /checkout` tiếp tục sang `payment-service`, service phía sau không tự biết request này thuộc Trace ID nào hoặc span nào phía upstream là parent. Để giữ request trong cùng trace, các thông tin đó phải được truyền kèm theo request. Cơ chế này được gọi là trace context propagation.

Trace Context Propagation Across Services

Khi request đi từ một operation sang một service khác, request cần mang theo *trace context* để downstream service có thể tiếp tục trace hiện tại thay vì tạo một trace độc lập.

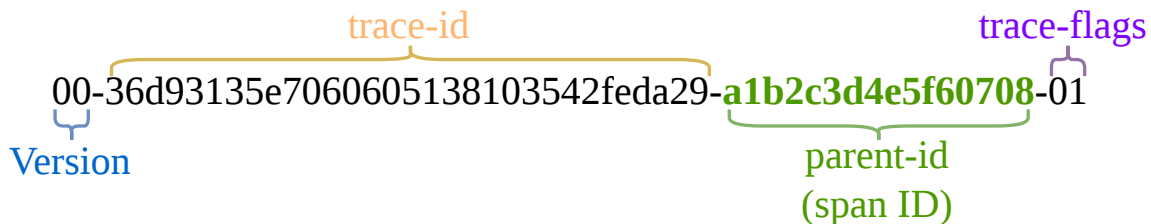
Trong quá trình xử lý `GET /checkout`, một outbound request được gửi đến `payment-service`.

Trace context được truyền cùng request này để **payment-service** biết request vẫn thuộc trace hiện tại và span phía upstream nào là parent. Sau khi đọc context, **payment-service** giữ nguyên Trace ID nhưng tạo một Span ID mới cho operation của mình. Nếu **payment-service** tiếp tục gọi external payment API, context hiện tại lại được truyền cùng request tiếp theo.

Quá trình này có thể hiểu theo bốn bước:

1. Request **GET /checkout** đang được xử lý trong một span.
2. Khi processing cần gọi **payment-service**, trace context được gắn vào outbound request.
3. **payment-service** đọc Trace ID và thông tin về span phía upstream.
4. **payment-service** tạo span mới với Span ID riêng nhưng vẫn sử dụng cùng Trace ID.

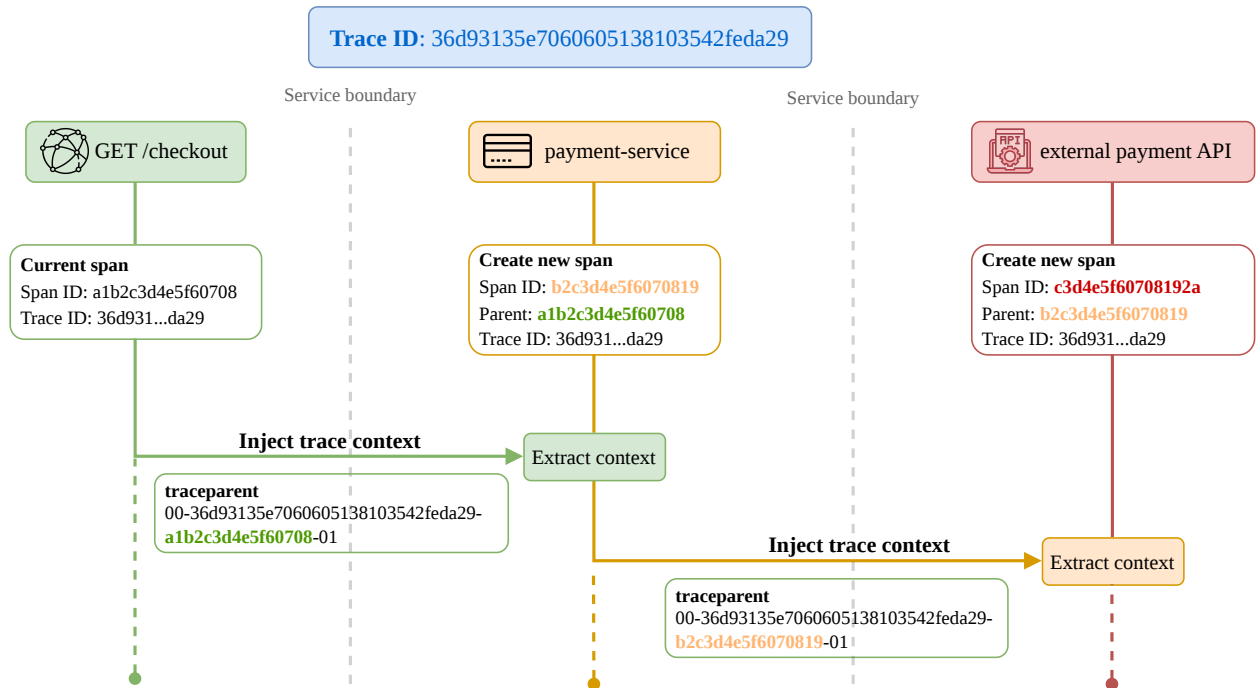
Trace context từ upstream được truyền cùng request để downstream service có đủ thông tin tạo span mới và nối span đó vào trace hiện tại. Với HTTP, thông tin này thường được truyền qua request header. Với các giao thức khác, context cũng có thể nằm trong gRPC metadata hoặc message headers/metadata. W3C Trace Context định nghĩa một format chung cho việc truyền trace identity, trong đó header **traceparent** chứa các thông tin cần thiết để tiếp tục trace.



Hình 40: Cấu trúc của header **traceparent** trong W3C Trace Context.

Header **traceparent** gồm bốn phần theo thứ tự **version**, **trace-id**, **parent-id** và **trace-flags**. Trong đó, **version** cho biết phiên bản của định dạng Trace Context; **trace-id** là identifier của toàn bộ trace và được giữ nguyên khi request đi qua các service; **parent-id** là Span ID của operation phía upstream đã gửi request hiện tại, nhờ đó downstream service biết span mới cần được nối với operation nào trước đó; còn **trace-flags** mang các cờ tracing, chẳng hạn thông tin về sampling.

Khi **payment-service** nhận header này, nó giữ nguyên **trace-id**, sử dụng **parent-id** để xác định span phía upstream và tạo một Span ID mới cho span của mình. Nếu service gọi tiếp một downstream component, Span ID mới đó sẽ trở thành **parent-id** trong **traceparent** của request tiếp theo. W3C Trace Context cũng có thể sử dụng thêm header **tracestate** để mang thông tin bổ sung của tracing system hoặc vendor.



Hình 41: Trace context được inject vào request và extract tại downstream service khi request đi qua các service boundaries.

Trong flow này, **trace-id** được giữ nguyên xuyên suốt request path, còn **parent-id** thay đổi theo span đang gửi request. Khi **GET /checkout** gửi request sang **payment-service**, Span ID của **GET /checkout** trở thành **parent-id**. Sau khi **payment-service** tạo span mới và gọi tiếp **external payment API**, Span ID của **payment-service** lại trở thành **parent-id** trong request tiếp theo.

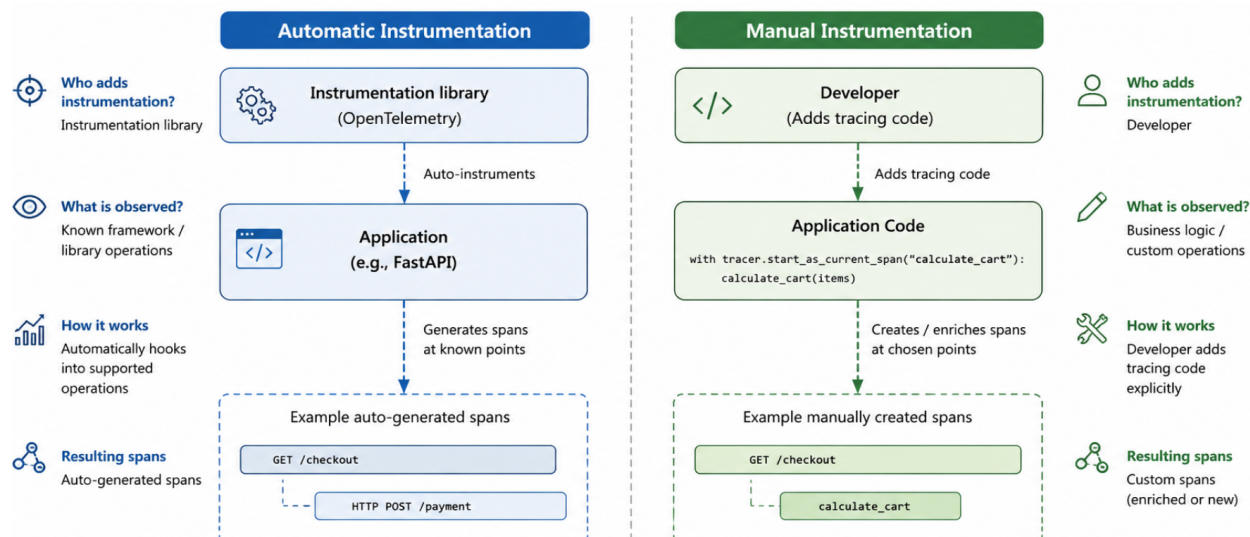
Nếu trace context bị mất tại một service boundary, phần execution phía sau có thể không còn được nối với trace ban đầu. Chẳng hạn, nếu **payment-service** không truyền context khi gọi **external payment API**, phần execution phía sau có thể xuất hiện như một trace độc lập thay vì tiếp tục trace của **/checkout**.

4.3 Instrumenting Applications with OpenTelemetry

Trong các application hiện đại, telemetry có thể được tạo ra từ nhiều framework, thư viện và service khác nhau. Nếu mỗi thành phần sử dụng một cách thu thập riêng, việc tạo và truyền telemetry sẽ khó đồng nhất trên toàn hệ thống. *Instrumentation*, tức quá trình gắn khả năng quan sát vào application, bổ sung các cơ chế để application tạo telemetry trong lúc chạy, chẳng hạn tạo spans, ghi attributes hoặc truyền trace context.

Application có thể được instrument bằng nhiều SDK hoặc agent khác nhau, bao gồm các giải pháp của Datadog, New Relic hay Dynatrace. Trong bài này, chúng ta tập trung vào OpenTelemetry (OTel) vì đây là một chuẩn mã nguồn mở, được sử dụng rộng rãi và có thể tích hợp với nhiều công cụ observability khác nhau. OTel cung cấp API, SDK và instrumentation libraries cho traces, metrics và logs. Trong distributed tracing, các thành phần này có thể tạo spans, gắn attributes và duy trì trace context giữa các service.

OpenTelemetry hỗ trợ hai cách chính để instrument application là *automatic instrumentation* và *manual instrumentation*. Automatic instrumentation tạo telemetry tại những điểm mà instrumentation component đã biết cách quan sát, còn manual instrumentation cho phép developer chủ động quyết định operation nào cần được theo dõi và bổ sung tracing logic vào code.



Hình 42: So sánh cách automatic instrumentation và manual instrumentation tạo spans trong application.

Automatic Instrumentation

Automatic instrumentation là cách telemetry được tạo tự động tại những điểm mà instrumentation component đã biết cách quan sát. Vì vậy, developer không cần tự thêm tracing logic cho từng operation.

Ví dụ với FastAPI, sau khi OpenTelemetry instrumentation được bật, nó có thể hook vào quá trình application tiếp nhận và xử lý HTTP request rồi tự tạo span tương ứng:

```
@app.get("/checkout")
async def checkout():
    return {"status": "ok"}
```

Request này vẫn có thể tạo ra một span cho GET /checkout dù function checkout() không chứa tracing logic riêng. Span đó có thể ghi nhận những thông tin như HTTP method, route, status code và thời gian xử lý.

Nếu application tiếp tục gọi payment-service thông qua một HTTP client đã được instrument, OpenTelemetry cũng có thể tự tạo span cho outbound request và truyền trace context sang service phía sau. Các spans được tạo theo cách này thường được gọi là *auto-generated spans*.

Manual Instrumentation

Automatic instrumentation chỉ tạo span tại những điểm mà instrumentation đã được thiết kế để quan sát. Vì vậy, nó có thể tự ghi nhận GET /checkout hoặc HTTP call đến payment-service,

nhưng một operation riêng bên trong application như `calculate_cart()` sẽ không tự trở thành một span.

Với manual instrumentation, developer chủ động bổ sung tracing logic vào code để quan sát những operation này. Developer có thể bổ sung thông tin vào một span hiện tại hoặc tạo thêm một span mới cho operation cần được theo dõi riêng.

Nếu span hiện tại đã tồn tại, kể cả span được tạo bằng automatic instrumentation, developer có thể lấy span đó và bổ sung thêm attributes, events hoặc status. Chẳng hạn, span của `GET /checkout` có thể đã được tạo tự động và application muốn bổ sung số lượng items trong cart:

```
from opentelemetry import trace

span = trace.get_current_span()
span.set_attribute("cart.item_count", len(items))
```

Trong trường hợp này, không có span mới được tạo. Developer chỉ bổ sung thêm context vào span hiện tại để trace chứa nhiều thông tin hữu ích hơn khi troubleshooting.

Nếu muốn theo dõi riêng một operation, developer có thể tạo một *custom span*, tức span tùy chỉnh. Ví dụ, bước tính toán giỏ hàng có thể được instrument như sau:

```
with tracer.start_as_current_span("checkout.calculate_cart") as span:
    span.set_attribute("cart.item_count", len(items))
    total = calculate_cart(items)
```

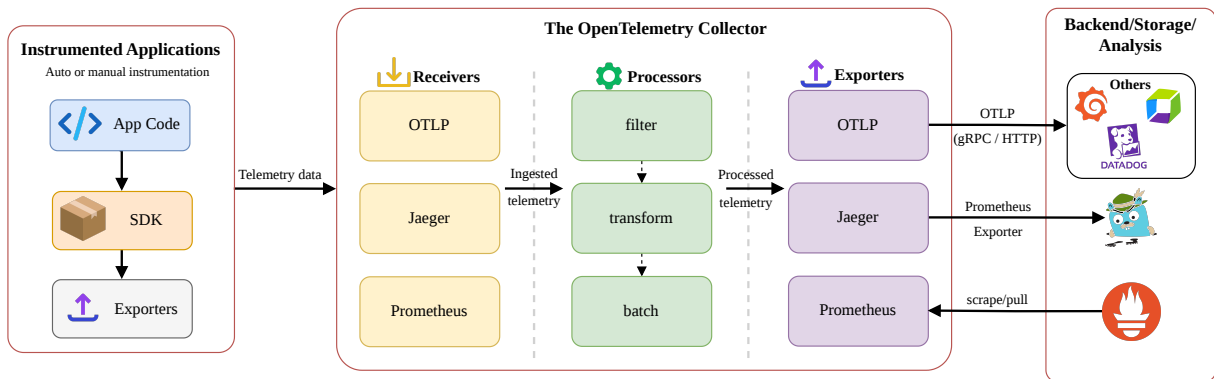
Ở đây, `checkout.calculate_cart` là một span mới được tạo riêng cho operation tính toán giỏ hàng. Vì span được tạo trong context hiện tại, nó có thể trở thành child span của `GET /checkout` và tiếp tục sử dụng cùng Trace ID.

Custom span giúp tracing system ghi nhận riêng duration, attributes và trạng thái của một bước xử lý thay vì chỉ thấy thời gian tổng của toàn request. Nó không phải một phương pháp instrumentation riêng mà là span do developer chủ động tạo khi sử dụng manual instrumentation.

Trong thực tế, automatic instrumentation và manual instrumentation thường được kết hợp với nhau. Automatic instrumentation tạo các spans cơ bản tại những điểm đã được hỗ trợ sẵn, còn manual instrumentation cho phép developer bổ sung context vào span hiện tại hoặc tạo thêm custom spans ở những operation cần quan sát sâu hơn.

4.4 Trace Pipeline with OTel Collector and Jaeger

Application có thể gửi telemetry trực tiếp đến backend. Tuy nhiên, khi hệ thống có nhiều service, mỗi service sẽ phải tự quản lý nơi gửi dữ liệu và cách export telemetry. Khi đó, số lượng kết nối point-to-point cũng tăng theo, khiến việc quản lý telemetry trên toàn hệ thống trở nên khó đồng nhất. Để giảm sự phân tán này, OpenTelemetry Collector được đặt giữa applications và backends, giúp tập trung việc tiếp nhận, xử lý và chuyển tiếp telemetry.



Hình 43: Kiến trúc tổng quát của telemetry pipeline từ instrumented applications qua OpenTelemetry Collector đến các backend lưu trữ và phân tích.

Trong pipeline trên, application tạo telemetry và gửi dữ liệu đến Collector. Collector tiếp nhận telemetry qua Receivers, xử lý qua Processors rồi chuyển dữ liệu ra ngoài bằng Exporters đến các backend phù hợp. Với distributed tracing, OTLP thường được dùng để truyền spans, còn các tracing backend như Jaeger đảm nhiệm việc lưu trữ, truy vấn và visualize traces.

OTLP (OpenTelemetry Protocol)

OTLP là giao thức dùng để truyền telemetry giữa các component trong hệ sinh thái OpenTelemetry. Với distributed tracing, OpenTelemetry SDK trong application có thể export spans bằng OTLP qua gRPC hoặc HTTP. Collector cũng có thể sử dụng OTLP để gửi telemetry đến backend hoặc component khác có hỗ trợ giao thức này. Vì vậy, OTLP là giao thức vận chuyển telemetry giữa các component, không phải một component riêng trong architecture.

OpenTelemetry (OTel) Collector

Collector là một service độc lập dùng để nhận, xử lý và chuyển tiếp telemetry. Architecture của Collector được xây dựng từ ba loại component:

Receiver → Processor → Exporter

- **Receiver** (tạm dịch: thành phần tiếp nhận) là entry point của telemetry vào Collector. Ví dụ, OTLP Receiver nhận telemetry được gửi bằng OTLP, trong khi Prometheus Receiver có thể scrape metrics từ các targets.
- **Processor** (tạm dịch: thành phần xử lý) xử lý telemetry sau khi nhận, với các ví dụ như `batch`, `filter` và `transform`.
- **Exporter** (tạm dịch: thành phần xuất dữ liệu) gửi telemetry đã xử lý đến backend hoặc một hệ thống khác.

Tùy vào pipeline, bước processing có thể được lược bỏ và telemetry được chuyển trực tiếp từ Receiver sang Exporter.

Việc khai báo Receiver, Processor hoặc Exporter riêng lẻ chưa tạo thành một pipeline hoạt động. Phần `service.pipelines` trong cấu hình Collector xác định các component được sử dụng và cách chúng được nối với nhau. Mỗi signal như traces, metrics hoặc logs có thể có pipeline riêng, chẳng hạn một trace pipeline được cấu hình như sau:

```
receivers:
  otlp:
    protocols:
      grpc:
      http:

processors:
  batch:

exporters:
  otlp:
    endpoint: tracing-backend:4317

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [otlp]
```

Trong cấu hình này, pipeline traces nhận spans qua OTLP, dùng `batch` để gom dữ liệu trước khi gửi và dùng exporter để chuyển traces đến backend. Collector không tạo lại Trace ID, Span ID hay parent-child relationship vì các thông tin này đã được hình thành từ quá trình instrumentation ở application.

Jaeger

Jaeger là một distributed tracing backend dùng để lưu trữ, tìm kiếm và visualize traces. Sau khi nhận các spans, Jaeger có thể tái dựng request path dựa trên Trace ID, Span ID và parent-child relationship giữa các spans.

Khi mở một trace, hai cách biểu diễn thường gặp là trace tree và timeline. Trace tree thể hiện cấu trúc parent-child giữa các operation, còn timeline cho biết thời điểm bắt đầu và duration của từng span. Nhờ đó, engineer có thể theo dõi request xuyên qua nhiều service, so sánh thời gian thực thi giữa các operation và khoanh vùng những span có khả năng liên quan đến latency hoặc failure.